華東理工大學

计算机图形学

2023年12月

奉贤校区

華東理工大學

10

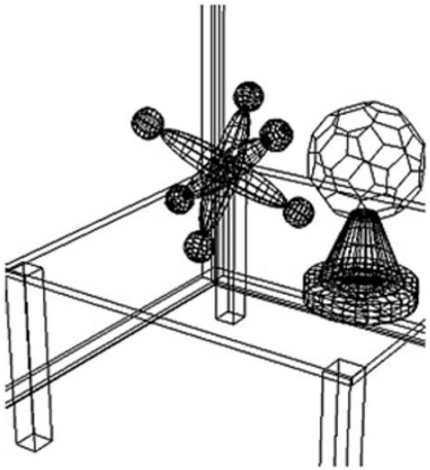
真实感图形绘制

Photorealistic Rendering

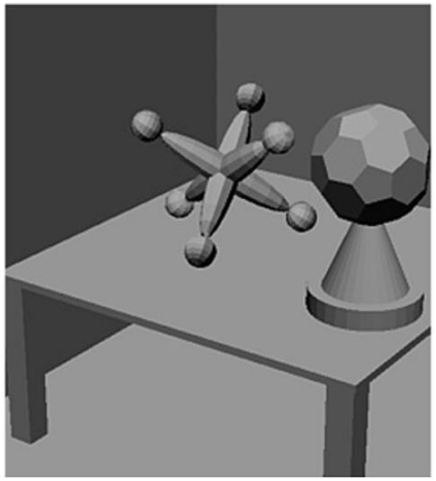
主要内容

- 基本概念
- 简单光照模型
- 基于简单光照模型的多边形绘制
- 模拟景物表面细节（纹理映射）
- 整体光照模型
- 光线追踪

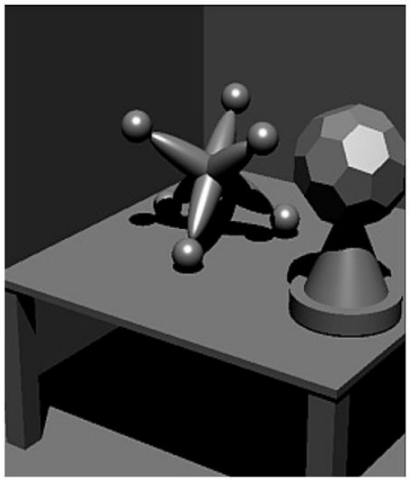
Rendering（渲染、绘制）



(a)

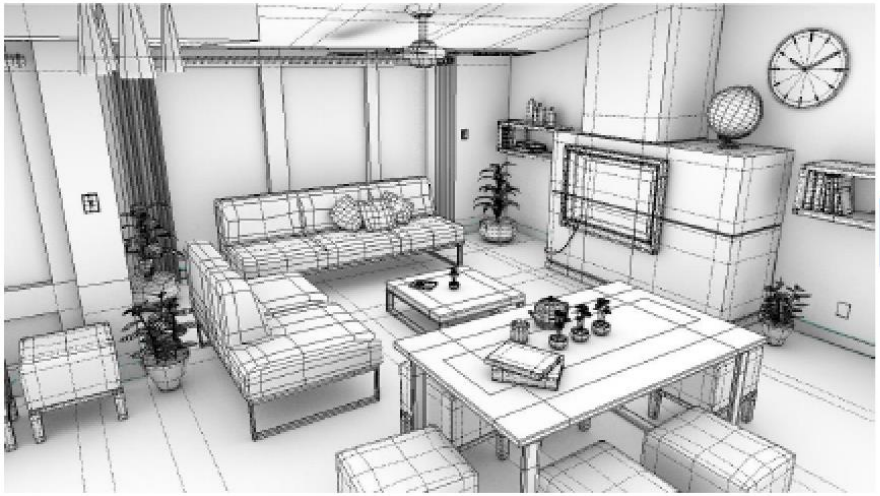


(b)



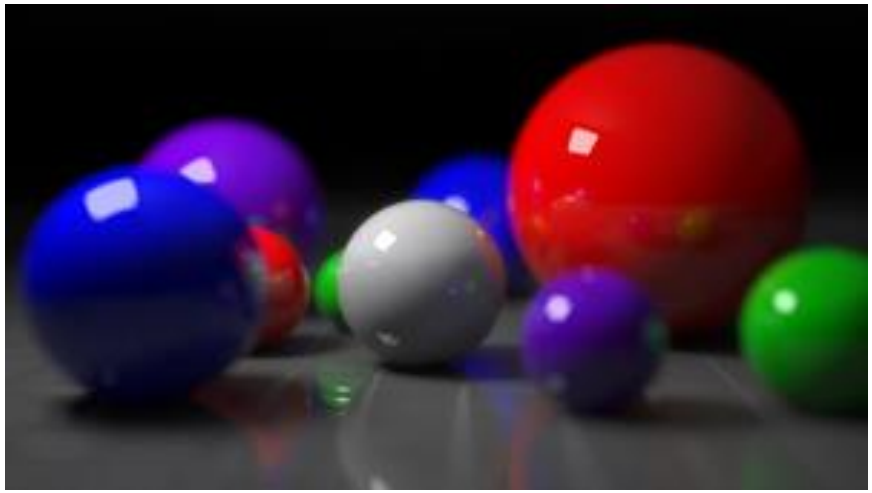
(c)

(d)



Photorealistic Rendering

Realistic Rendering



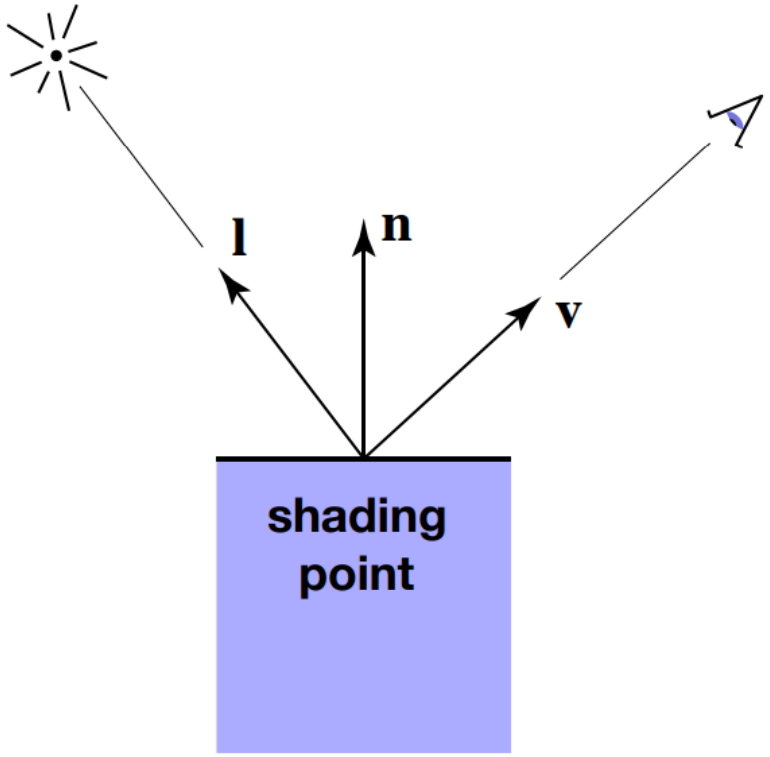
Recap: Local Shading Model

Credit by: Lingqi Yan

Local Shading

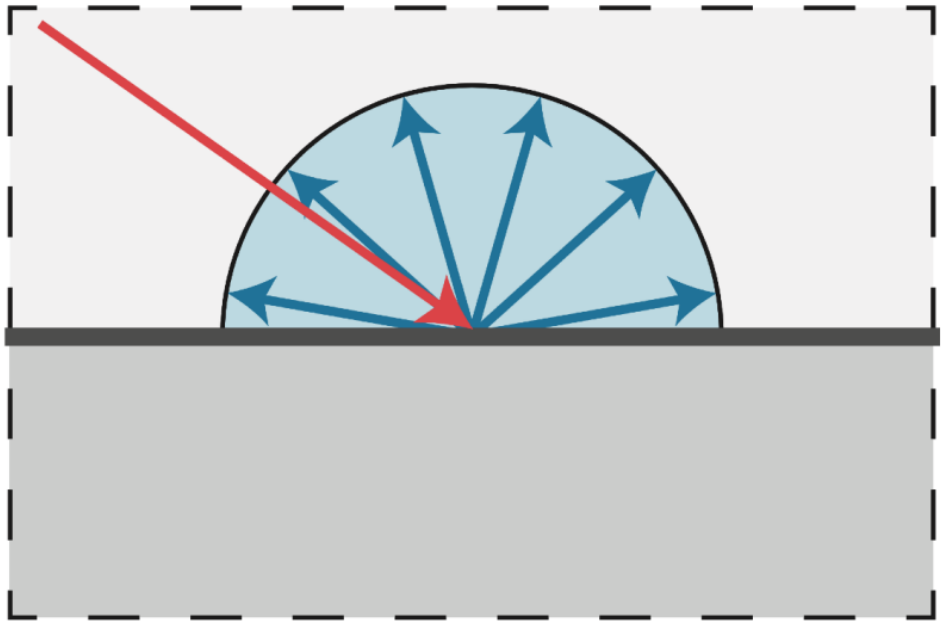
Inputs:

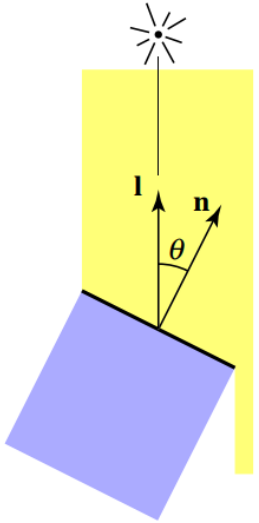
- Viewer direction, v
- Surface normal, n
- Light direction, l
  (for each of many lights)
- Surface parameters
  (color, shininess, …)



Diffuse Reflection

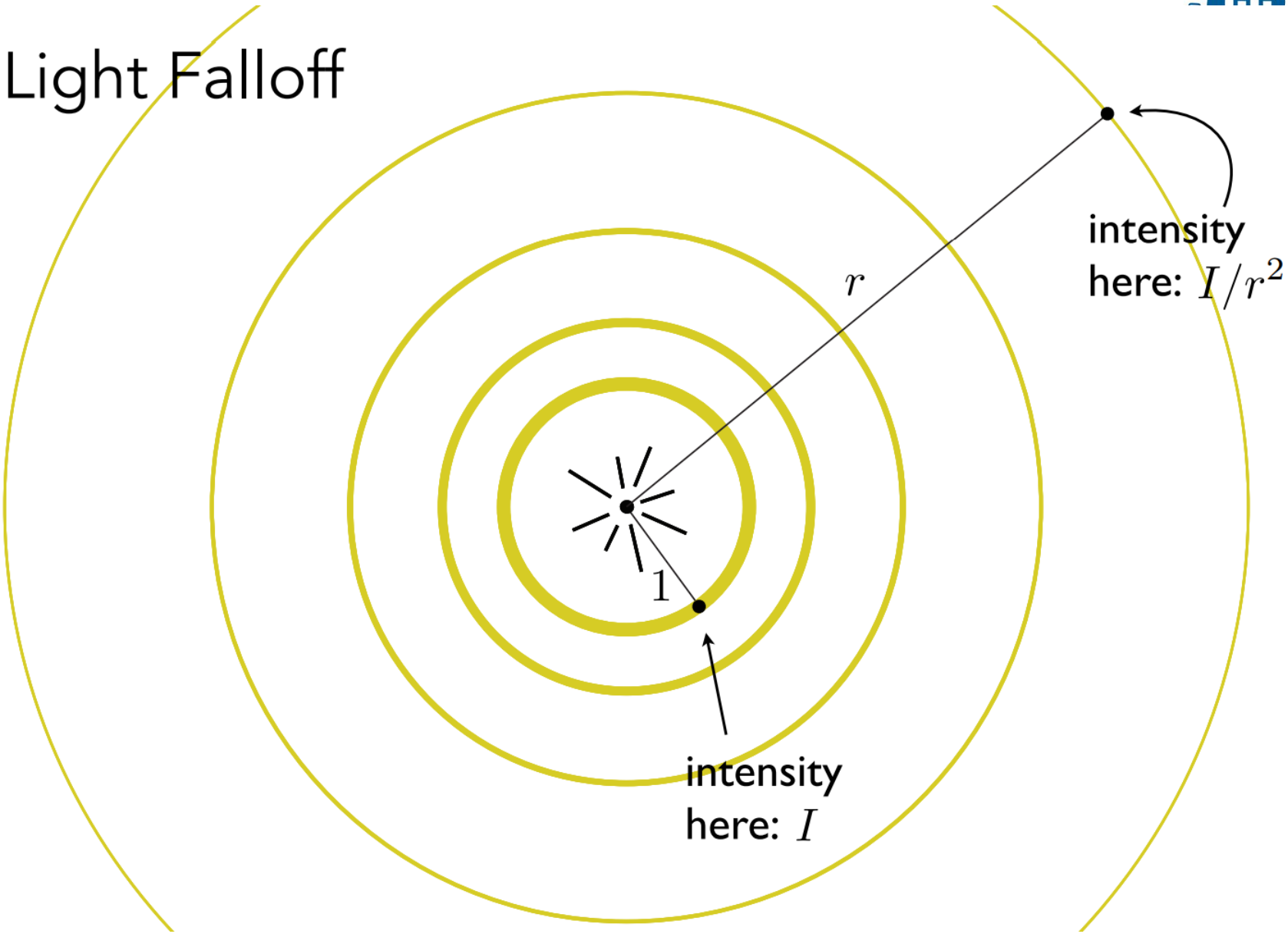
- Light is scattered uniformly in all directions
  - Surface color is the same for all viewing directions





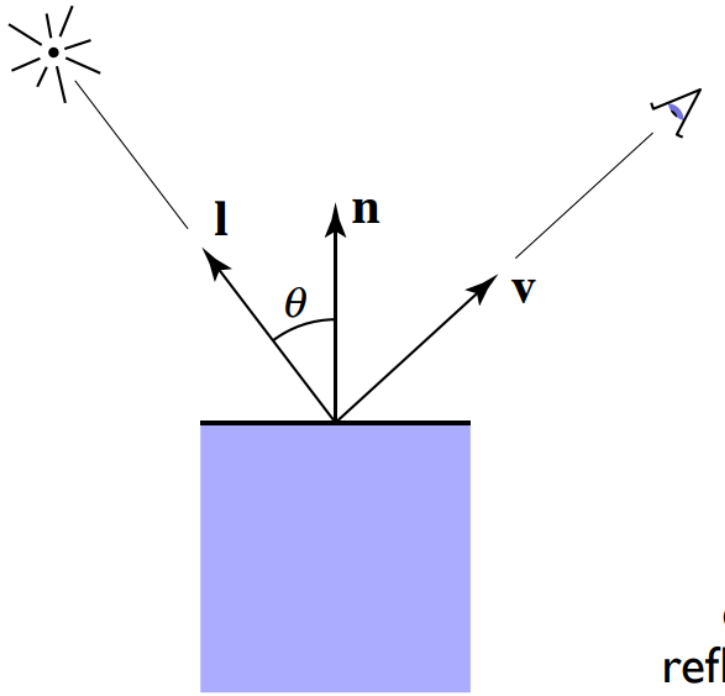
In general, light per unit area is proportional to $\cos\theta = l \cdot n$

Light Falloff



Lambertian (Diffuse) Shading

- Shading independent of view direction



energy arrived
at the shading point

$$L_d = k_d\,(I/r^2)\max(0, \mathbf{n}\cdot\mathbf{l})$$

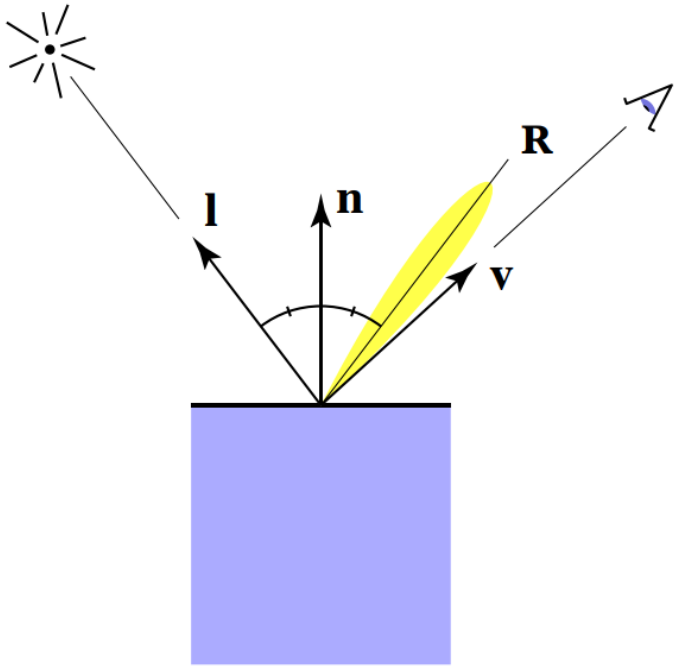
diffusely
reflected light

diffuse
coefficient
(color)

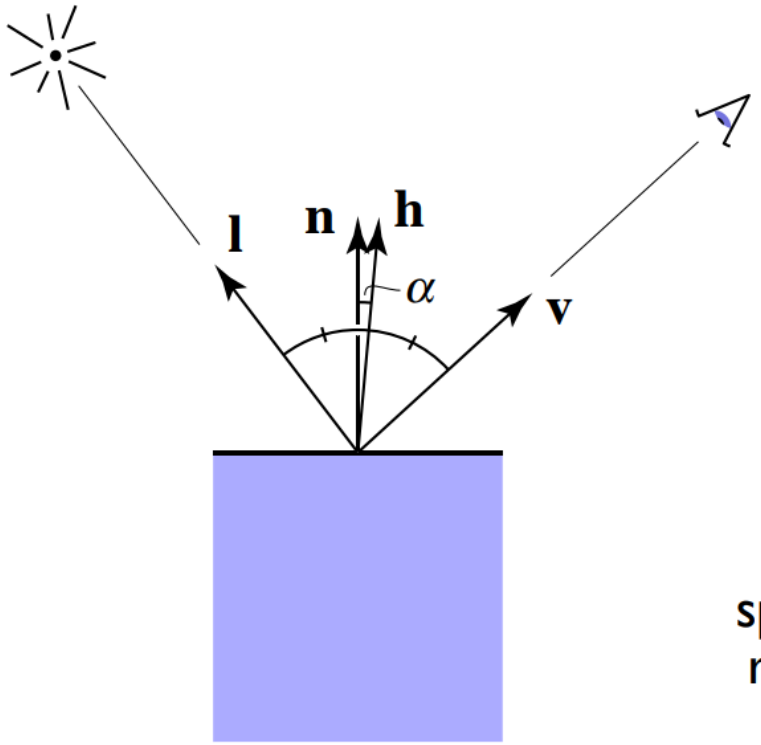
energy received
by the shading point

Specular Term

- Intensity depends on view direction
  - Bright near mirror reflection direction



Specular Term (Blinn-Phong，改进版Phong模型)



$$\mathbf{h} = \text{bisector}(\mathbf{v}, \mathbf{l})$$

(半程向量)

$$= \frac{\mathbf{v} + \mathbf{l}}{\|\mathbf{v} + \mathbf{l}\|}$$

$$L_s = k_s \left(I/r^2\right) \max(0, \cos\alpha)^p$$

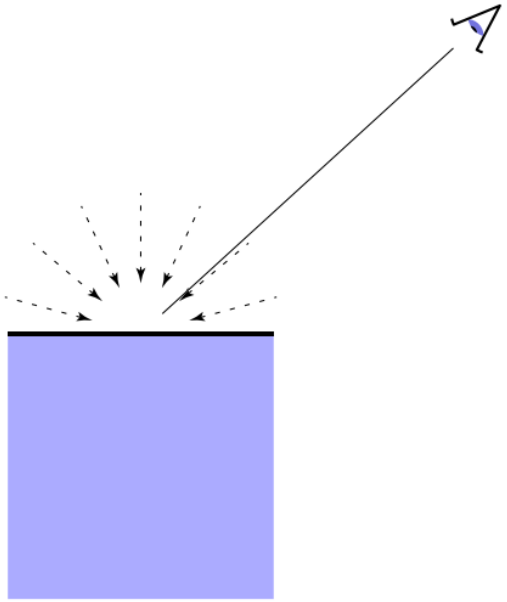
$$= k_s \left(I/r^2\right) \max(0, \mathbf{n} \cdot \mathbf{h})^p$$

specularly reflected light

specular coefficient

Ambient Term

- Shading that does not depend on anything
  - Add constant color to account for disregarded illumination and fill in black shadows

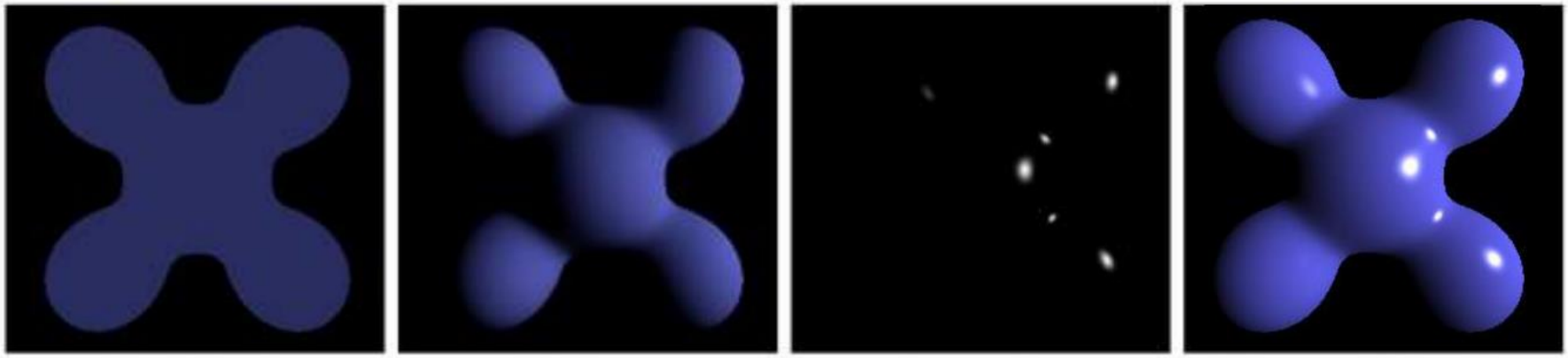


$$L_a = k_a I_a$$

ambient coefficient

reflected ambient light

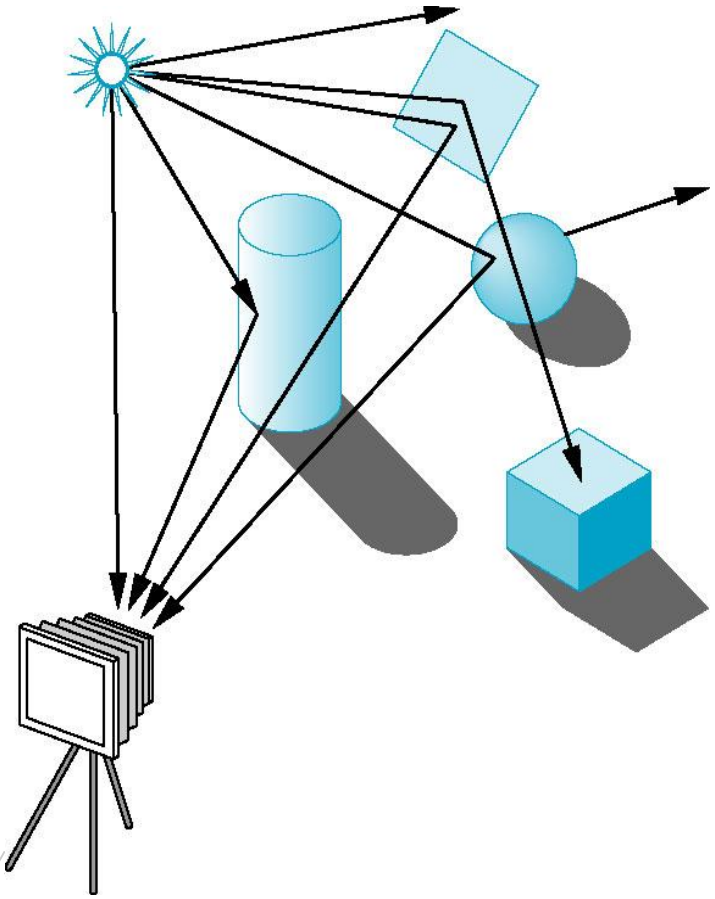
Blinn-Phong Shading Model



$$L = L_a + L_d + L_s$$
$$= k_a\, I_a + k_d\, (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s\, (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{h})^p$$

局部光照模型的不足

- 过于简化的反射模型：复杂材质
- 间接反射光
- 其他
  - 遮挡
  - 阴影
  - …



材质：物体对光的视觉反应

Material = BRDF

Bidirectional Reflectance Distribution Function

简化使用时为常数，实际上是个分布函数

整体光照模型与光线跟踪

1 整体光照模型

2 渲染方程

3 光线跟踪算法

整体光照模型

- 一个完整的光照明模型应该包括由光源和环境引起的漫反射分量、镜面反射分量、规则透射分量以及漫透射分量等。
  - 仅考虑由光源引起的漫反射分量和镜面反射分量，而环境反射分量则简单地用一常数来代替，这类光照模型称为局部光照模型。
  - 能同时模拟光源和环境照明效果的光照模型称为整体光照模型。

Whitted光照模型

- Whitted在简单光照模型中增加了环境镜面反射光和环境规则透射光，以模拟周围环境的光投射在景物表面上产生的理想镜面反射和规则透射现象。

$$I = I_{local} + K_s I_s + K_t I_t$$

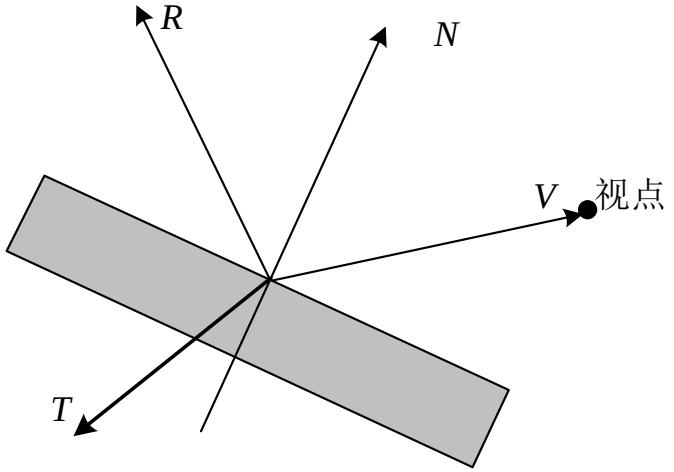
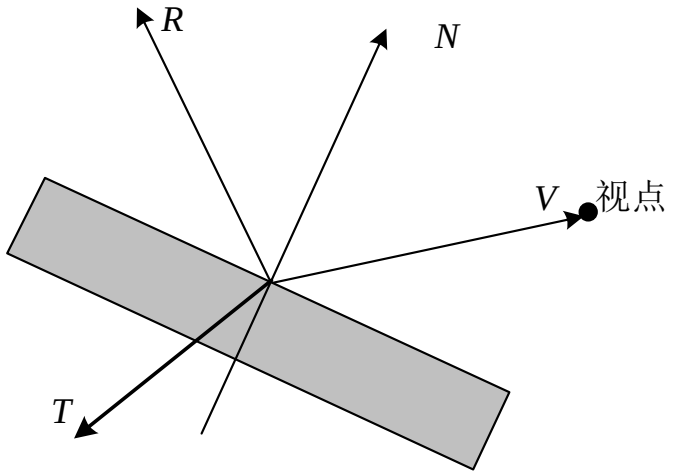


图10.13 物体表面的镜面反射和投射

Whitted光照模型

$$I = I_{local} + K_s I_s + K_t I_t$$



I_{local}：由光源直接照射在表面上引起的反射光亮度

I_s：沿V的镜面反射方向R入射到表面上的环境光在表面上产生的镜面反射光

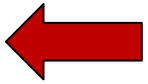
I_t：沿V的规则透射方向T入射到表面上的环境光通过透射在表面上产生的规则透射光

k_s：表面的镜面反射率

k_t：表面的透射率

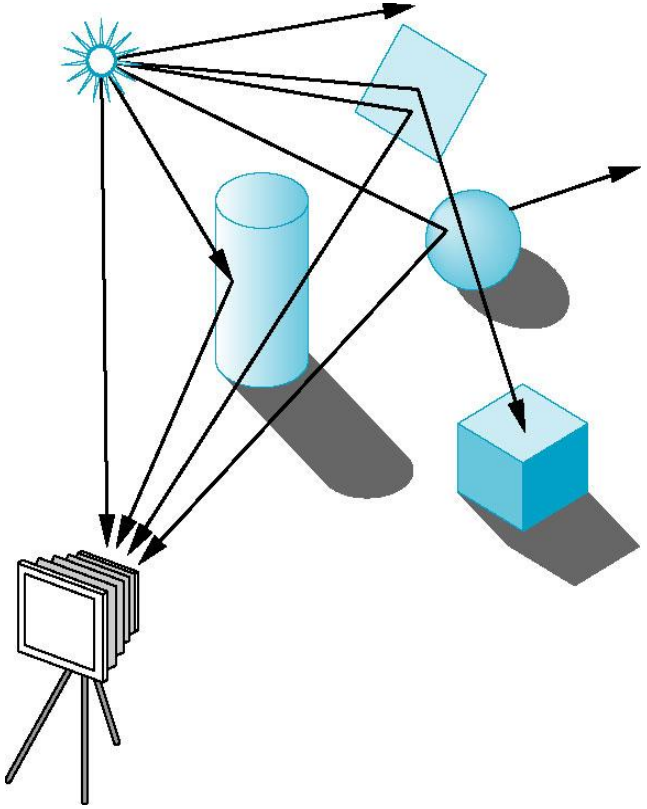
整体光照模型与光线跟踪

1. 整体光照模型
2. 渲染方程
3. 光线跟踪算法



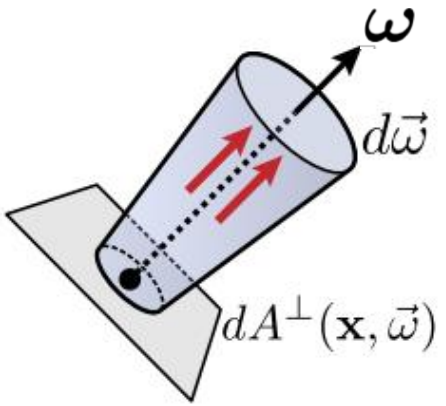
光路的不断反射传播

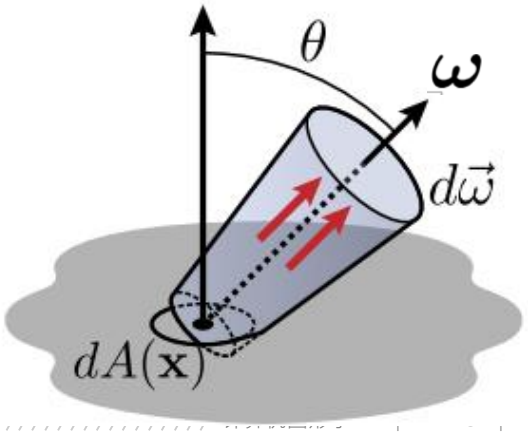
- 1. Light travels in straight lines (though this is wrong)
- 2. Light rays do not “collide” with each other if they cross (though this is still wrong)
- 3. Light rays travel from the light sources to the eye (but the physics is invariant under path reversal - reciprocity).



Radiance（辐射率）

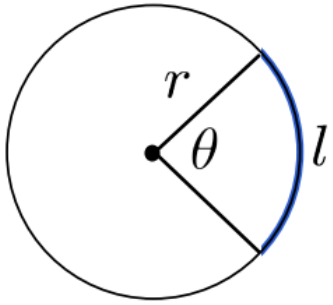
- Radiant energy at ***x*** in direction ***ω***:
  - A 5D function $L(x, \omega)$: Power
    - per projected surface area
    - per solid angle





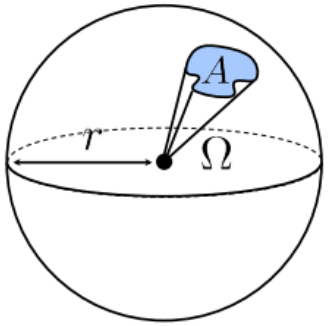
Solid Angle

- Angle
  - circle: 2π radians

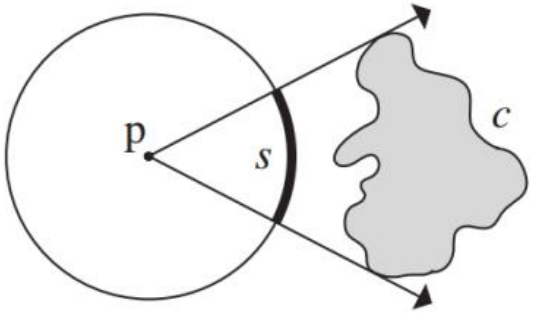


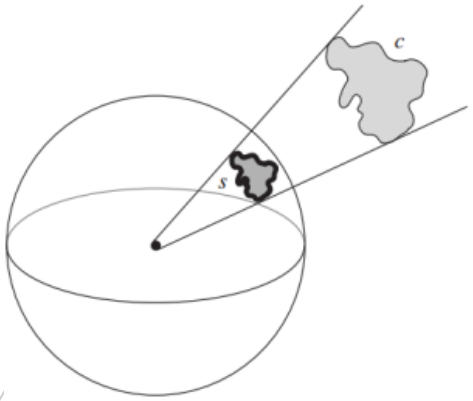
$$\theta = \frac{l}{r}$$

- Solid angle
  - sphere: 4π steradians



$$\Omega = \frac{A}{r^2}$$



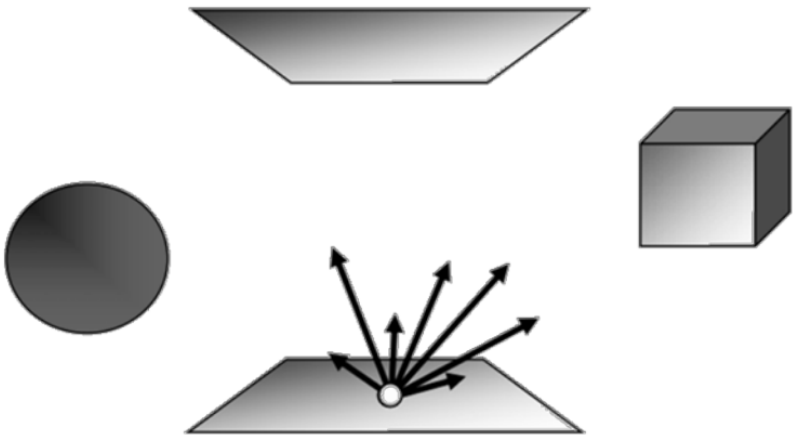


Light Transport

- Goal
  - Describe steady-state radiance distribution in virtual scenes
- Assumptions
  - Geometric optics
  - Achieves steady state instantaneously

Radiance at Equilibrium

- Radiance values at all points in the scene and in all directions expresses the equilibrium
  - 5D “Light-field”
- We only consider radiance on surfaces (4D)
  - Assuming no volumetric scattering or absorption



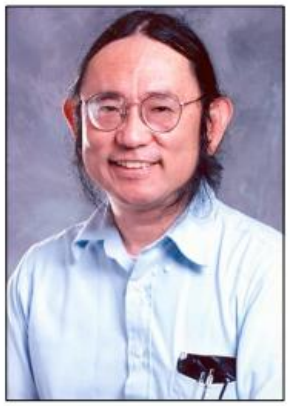
渲染方程Rendering Equation (RE)

- RE describes the distribution of radiance at equilibrium
- RE involves:
  - Scene geometry
  - Light source info.
  - Surface reflectance info.
  - Radiance values at all surface points in all directions

Known

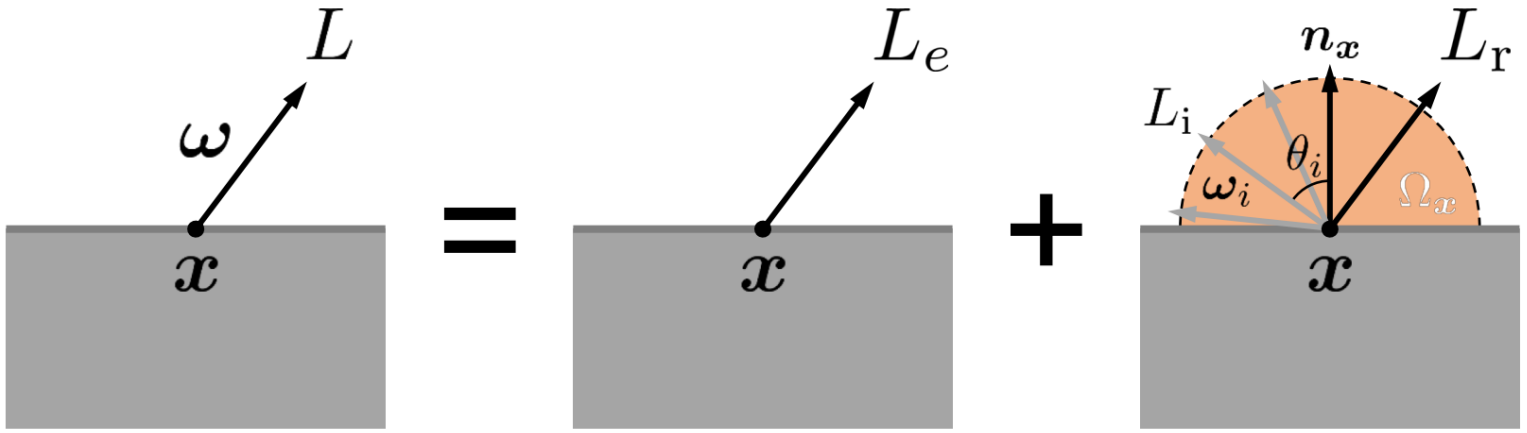
Unknown

Rendering Equation (RE)



Kajiya, 1986

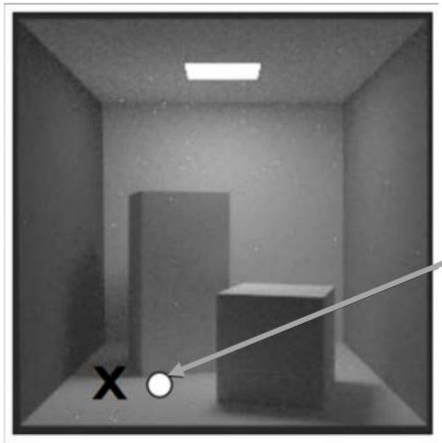
[*outgoing*] = [*emitted*] + [*reflected*]



$$L(\boldsymbol{x},\boldsymbol{\omega}) = L_e(\boldsymbol{x},\boldsymbol{\omega}) + L_r(\boldsymbol{x},\boldsymbol{\omega})$$

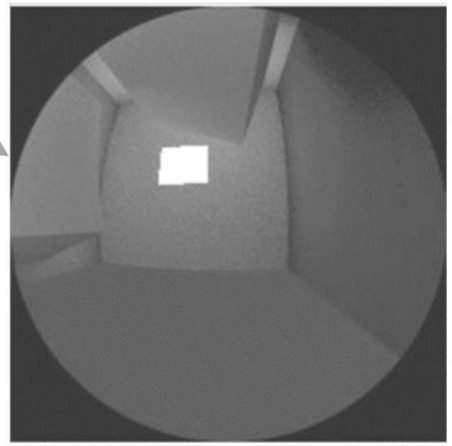
$$\int_{\Omega_{\boldsymbol{x}}} L_i(\boldsymbol{x},\boldsymbol{\omega}_i)\, f_r(\boldsymbol{x},\boldsymbol{\omega}_i \leftrightarrow \boldsymbol{\omega}) \underbrace{\langle \boldsymbol{n}_{\boldsymbol{x}},\boldsymbol{\omega}_i \rangle}_{=\cos\theta_i} \mathrm{d}\boldsymbol{\omega}_i$$

Rendering Equation



$$L(\boldsymbol{x},\boldsymbol{\omega}) = L_e(\boldsymbol{x},\boldsymbol{\omega}) + \int_{\Omega_{\boldsymbol{x}}} L_i(\boldsymbol{x},\boldsymbol{\omega}_i) f_r(\boldsymbol{x},\boldsymbol{\omega}_i \leftrightarrow \boldsymbol{\omega}) \langle \boldsymbol{n}_{\boldsymbol{x}},\boldsymbol{\omega}_i \rangle \mathrm{d}\boldsymbol{\omega}_i$$

Incoming radiance



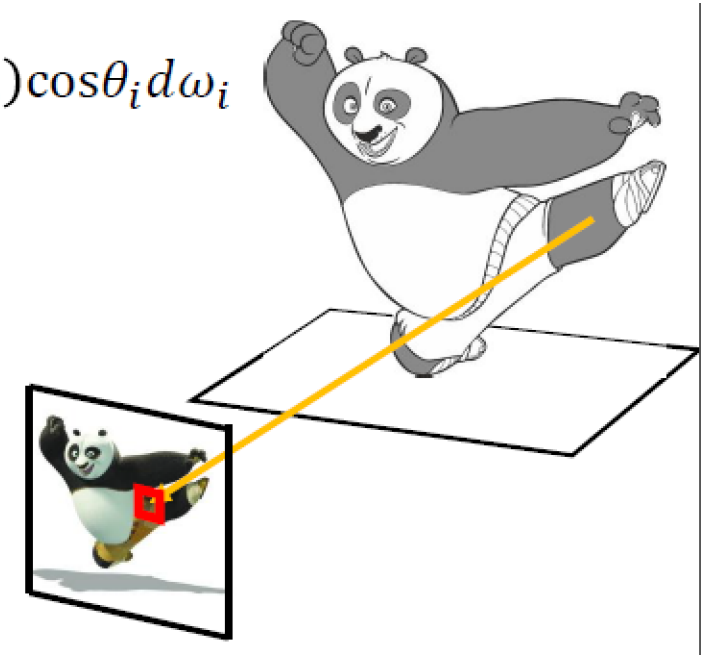
Solving Rendering Equation

Credit by Rui Wang, Shuang Zhao

Solving Rendering Equation

- What is the color of pixel (i,j)?

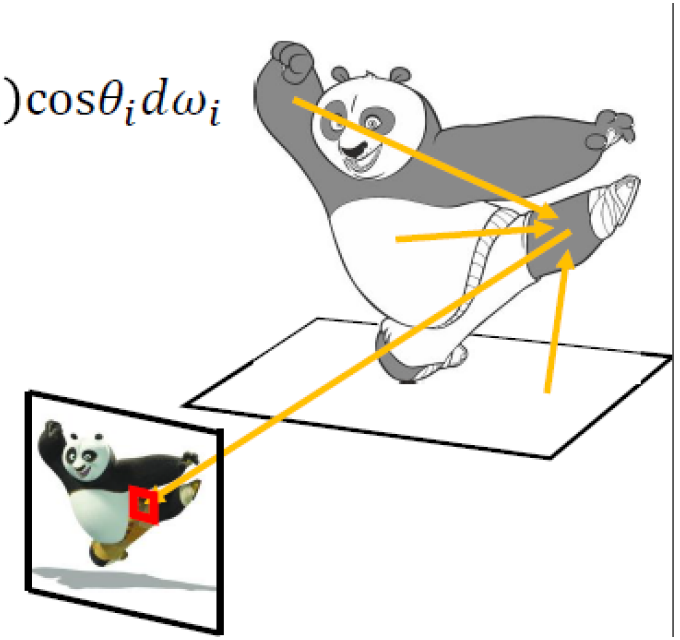
$$L_o(x,\omega_o) = L_e(x,\omega_o) + \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o) \cos\theta_i d\omega_i$$



Solving Rendering Equation

- What is the color of pixel (i,j)?

$$L_o(x,\omega_o) = L_e(x,\omega_o) + \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o)\cos\theta_i d\omega_i$$

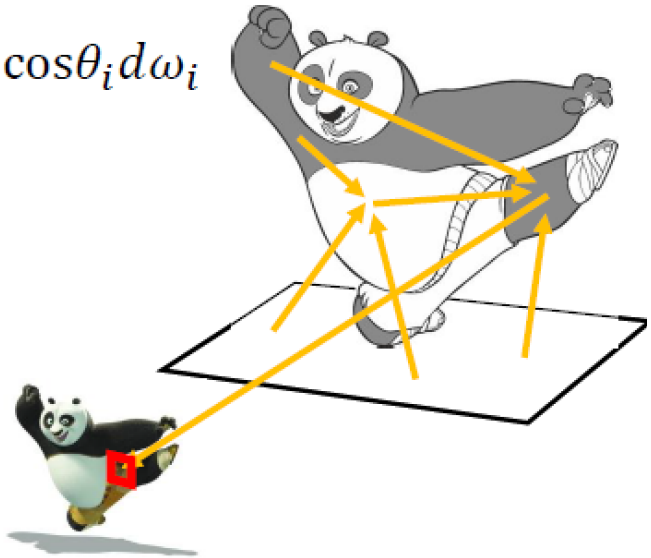


Solving Rendering Equation

- What is the color of pixel (i,j)?

$$L_o(x,\omega_o) = L_e(x,\omega_o) + \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o)\cos\theta_i d\omega_i$$

$$L_o(x,\omega_o) = \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o)\cos\theta_i d\omega_i$$



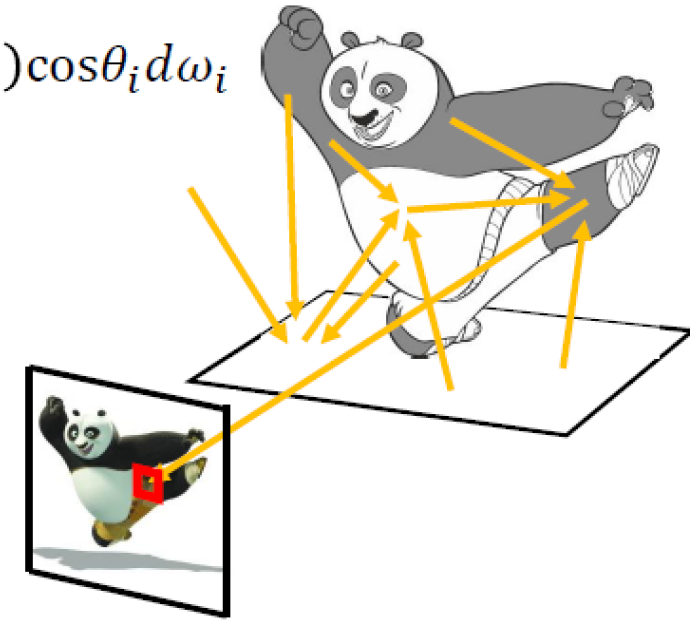
Solving Rendering Equation

- What is the color of pixel (i,j)?

$$L_o(x,\omega_o) = L_e(x,\omega_o) + \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o) \cos\theta_i d\omega_i$$

$$L_o(x,\omega_o) = \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o) \cos\theta_i d\omega_i$$

$$L_o(x,\omega_o) = \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o) \cos\theta_i d\omega_i$$



Solving Rendering Equation

- What is the color of pixel (i,j)?

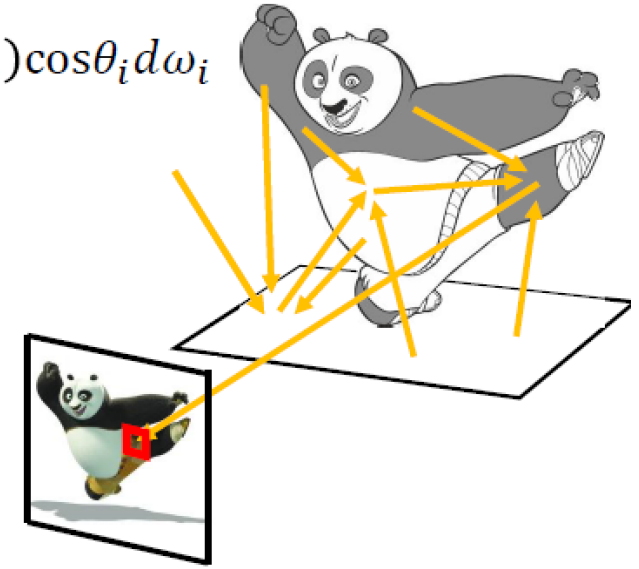
$$L_o(x,\omega_o) = L_e(x,\omega_o) + \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o)\cos\theta_i d\omega_i$$

$$L_o(x,\omega_o) = \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o)\cos\theta_i d\omega_i$$

$$L_o(x,\omega_o) = \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o)\cos\theta_i d\omega_i$$

$$\vdots$$

$$L_o(x,\omega_o) = \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o)\cos\theta_i d\omega_i$$

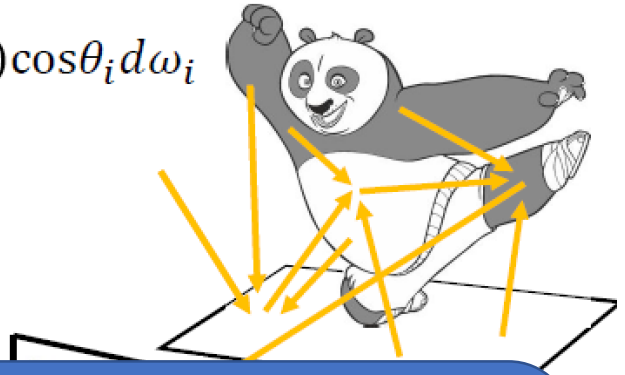


Solving Rendering Equation

- What is the color of pixel (i,j)?

$$L_o(x,\omega_o) = L_e(x,\omega_o) + \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o) \cos\theta_i d\omega_i$$

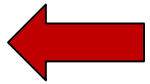
$$L_o(x,\omega_o) = \int_{H^2} L_i(x,\omega_i) f_r(x,\omega_i \rightarrow \omega_o) \cos\theta_i d\omega_i$$



Curse of dimensionality!

整体光照模型与光线跟踪

1. 整体光照模型
2. 渲染方程
3. 光线跟踪算法



光线跟踪算法

- **光线跟踪（Ray Tracing）**方法基于几何光学的原理，通过模拟光的传播路径来确定反射、折射和阴影等。

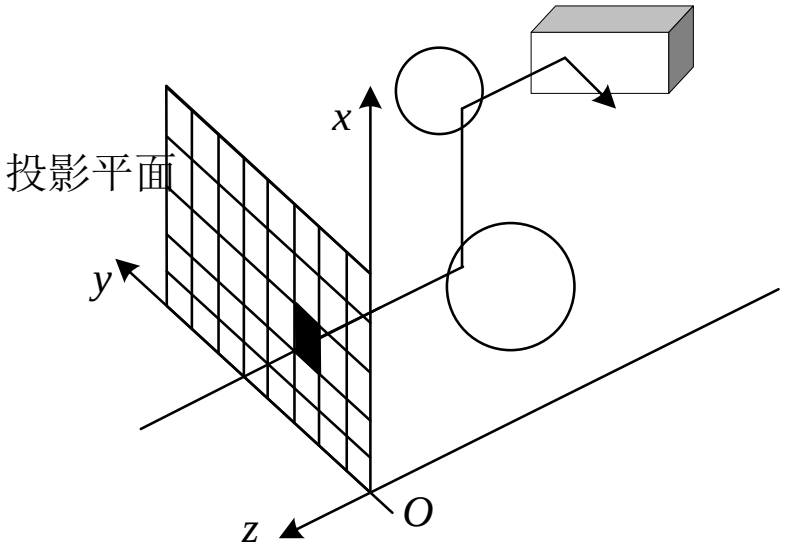


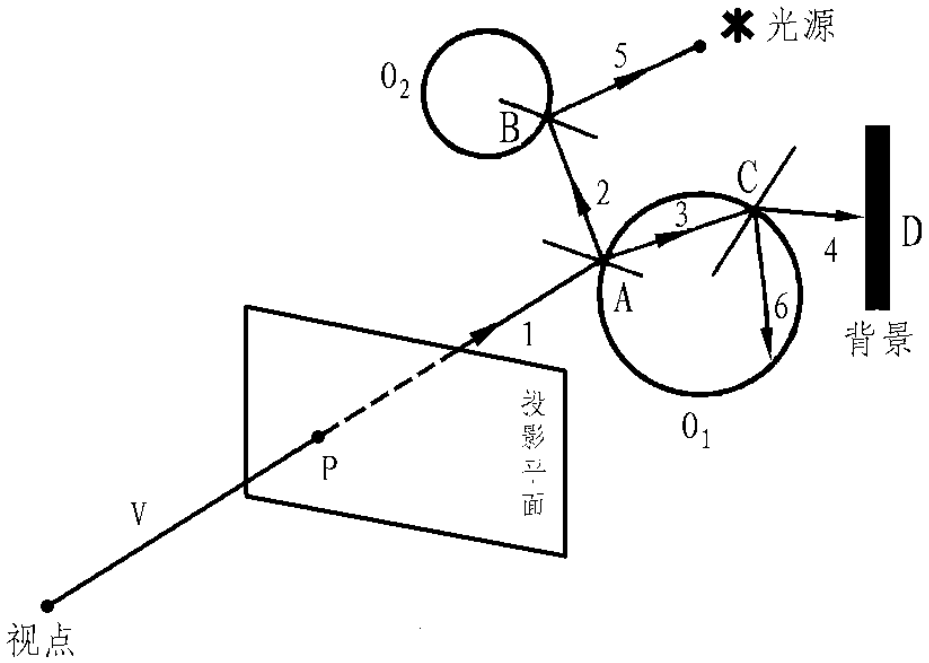
图10.14 光线跟踪算法

光线跟踪算法

- 迄今为止最为成功的生成真实感图形系列算法之一
  - 算法简单
  - 生成的图形真实感强
  - 计算量大
- 其前身是光线投射（Ray Casting）算法

理解光线跟踪

- 光线跟踪本质上是个递归算法，每个象素的光强度必须综合各级递归计算的结果才能获得。光线跟踪结束的条件有三个，光线与光源相交、光线与背景相交以及被跟踪的光线对第一个交点处的光强度作用趋近于0。



光线跟踪算法步骤

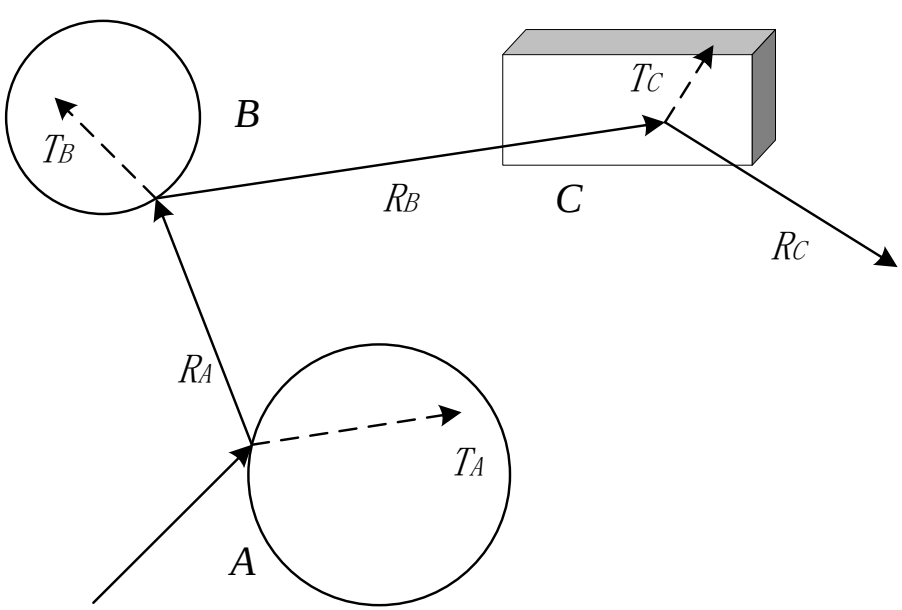
- 从视点出发通过该像素中心向场景发出一条光线R，并求出R与场景中物体的全部交点；获得离视点最近交点P；并依据局部光照明模型计算P处颜色值I_{local}（光线投射）；
- 在P处沿着R镜面反射方向和透射方向各衍生一条光线，若点P所在表面非镜面或不透明体，则无需衍生出相应光线。

光线跟踪算法步骤

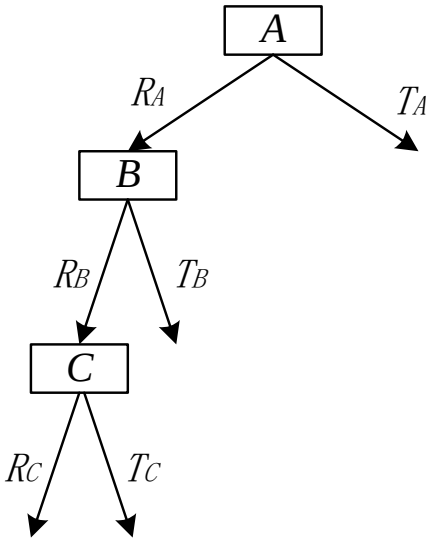
- 分别对衍生出的光线递归地执行前面步骤，计算来自镜面反射方向和透射方向上周围环境对点P光亮度的贡献Is和It；
- 依据Whitted光照明模型即可计算出点P处的光亮度，并将计算出的光亮度赋给该像素。

- 光线跟踪过程可用一棵二叉树（称为光线跟踪树）来表示。逐个将相交点加入到二叉树中，树的左分支表示反射光线，右分支表示透射光线。

光线跟踪算法步骤



(a) 光线在景物中反射和折射



(b) 二叉光线跟踪树

图10.15 光线跟踪算法

光线追踪的实现

- 固定管线
- 着色器编程
- WebGL

光线跟踪算法开源代码（POV－Ray）

- **http://www.povray.org**

"Thanks for all the fish"

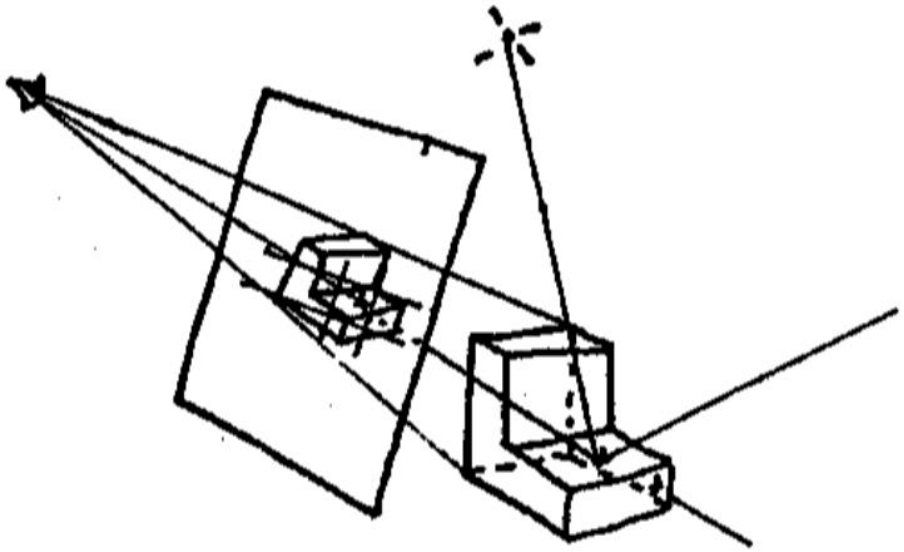
Lakehurst Disaster
兴登堡号空难

基于GLSL实现

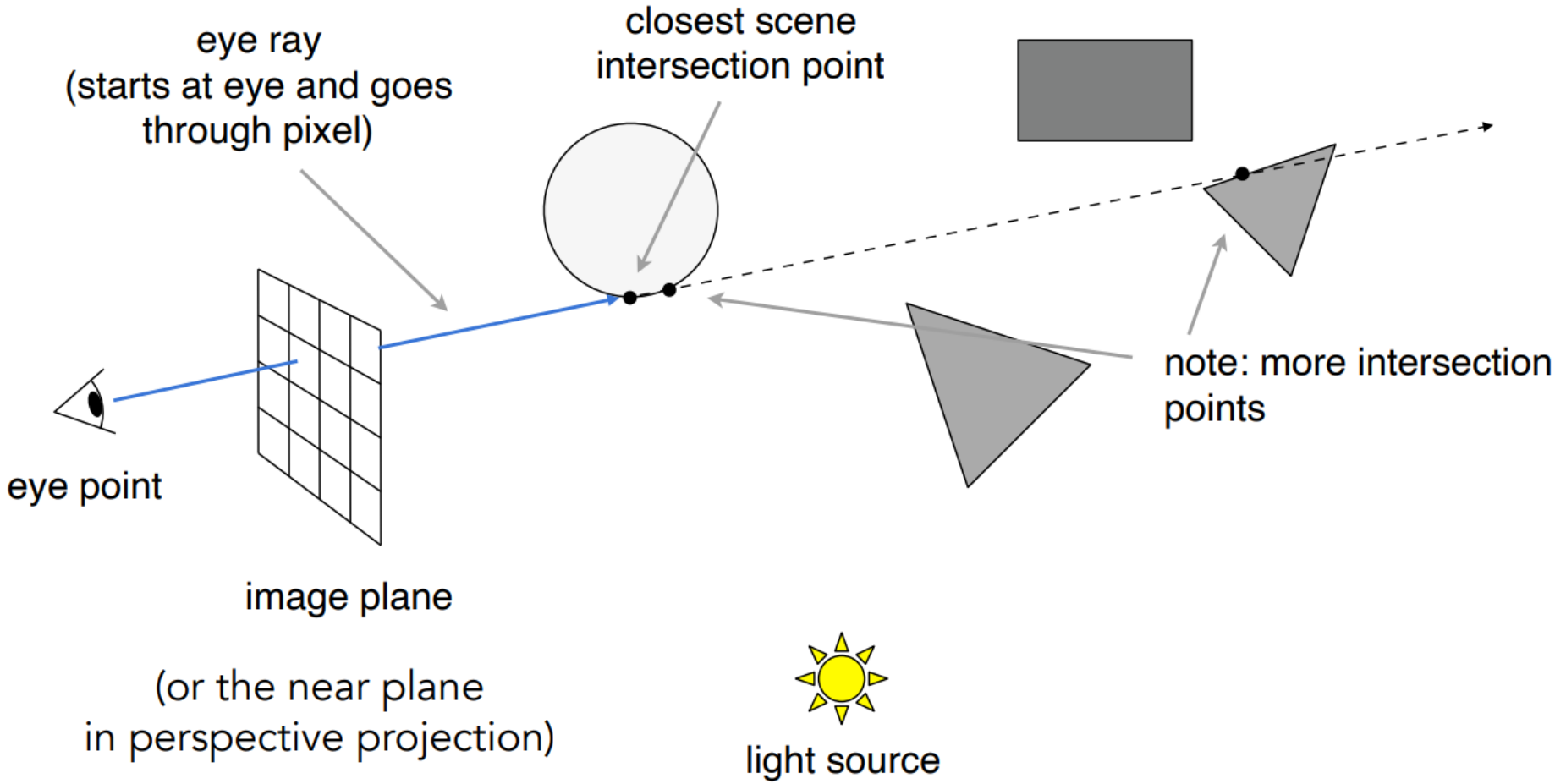
- Ray Casting
- 阴影
- 反射
- 折射
- 迭代次数

Ray Casting

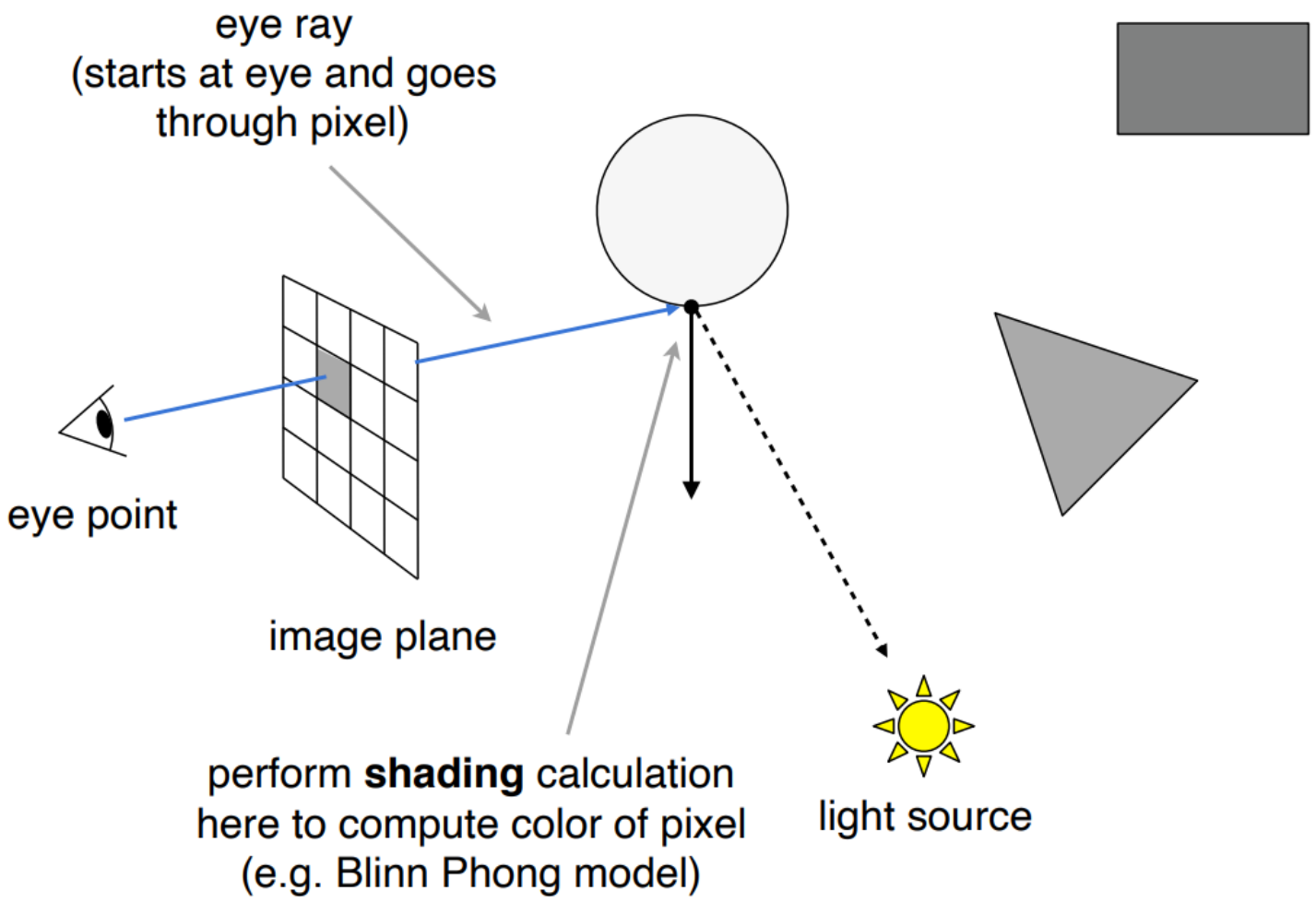
- Appel 1968 - Ray casting
  - 1. Generate an image by **casting one ray per pixel**
  - 2. Check for shadows by **sending a ray to the light**



Ray Casting - Generating Eye Rays



Ray Casting - Shading Pixels (Local Only)

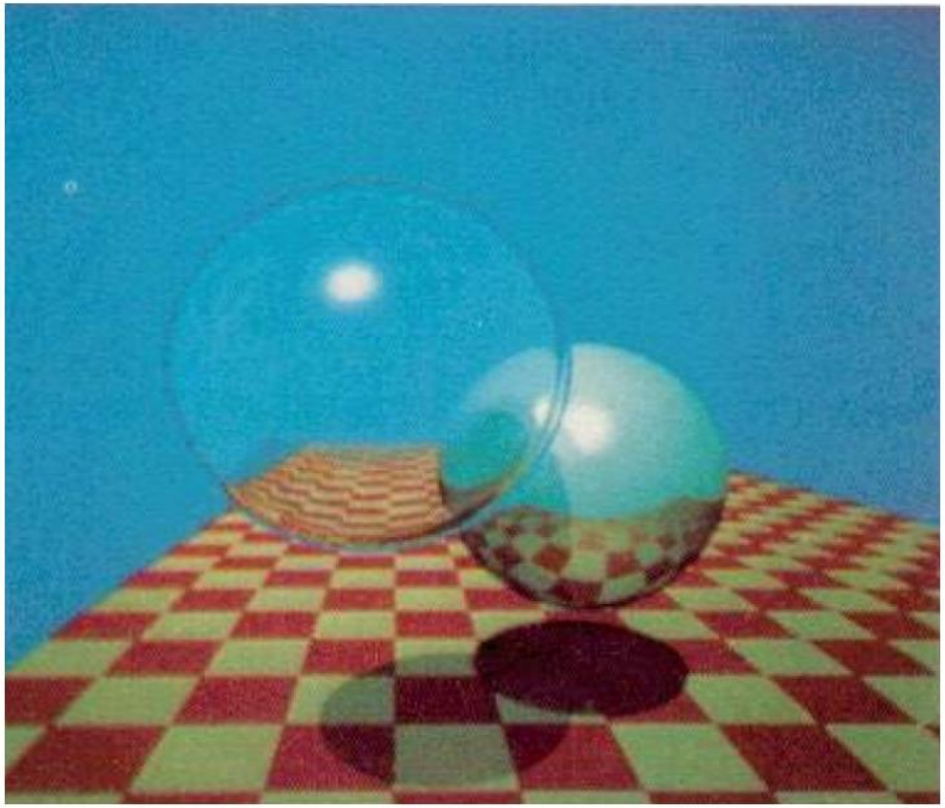


Recursive (Whitted-Style) Ray Tracing

"An improved Illumination model for shaded display"
T. Whitted, CACM 1980

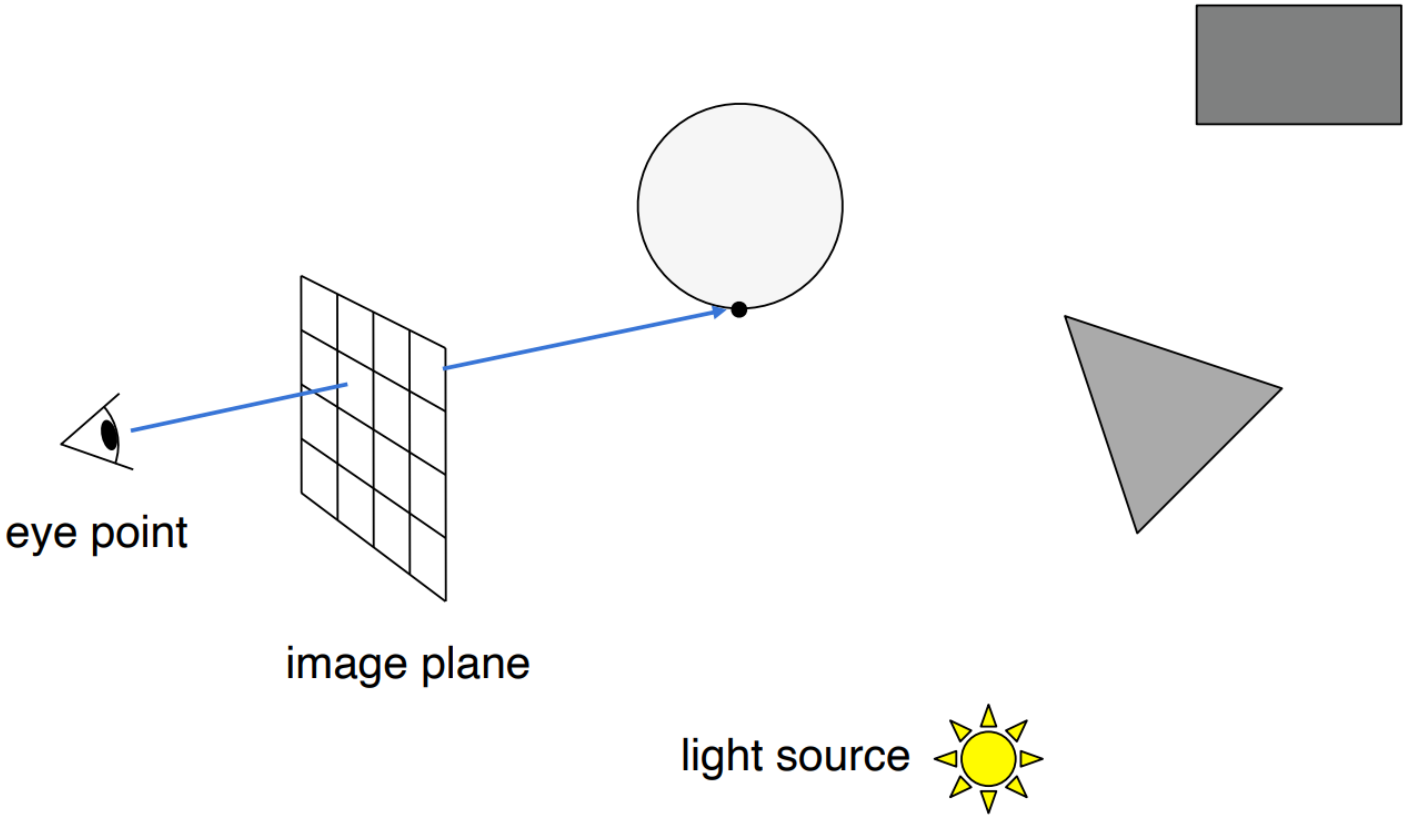
Time:

- VAX 11/780 (1979) 74m
- PC (2006) 6s
- GPU (2012) 1/30s

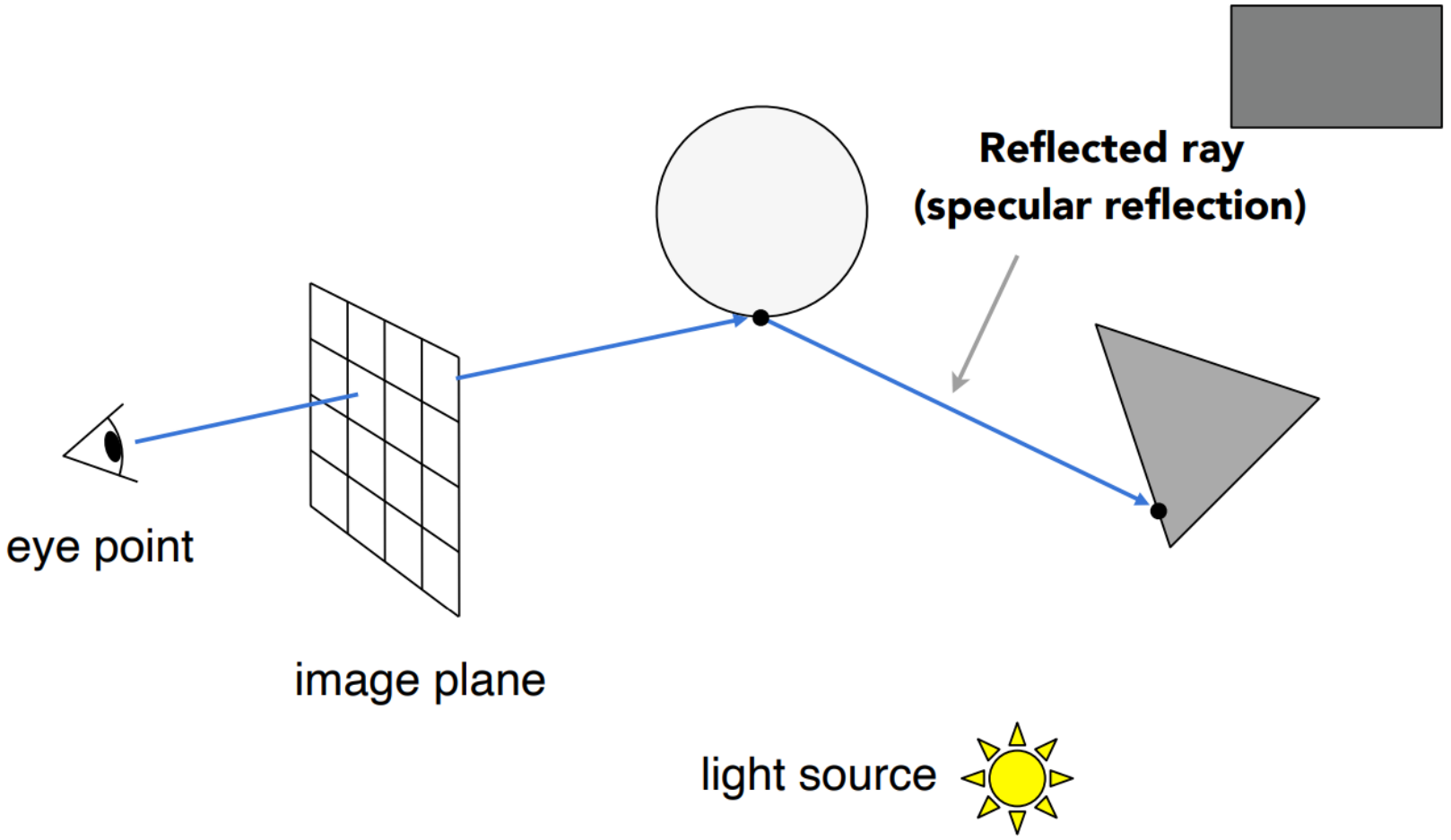


Spheres and Checkerboard, T. Whitted, 1979

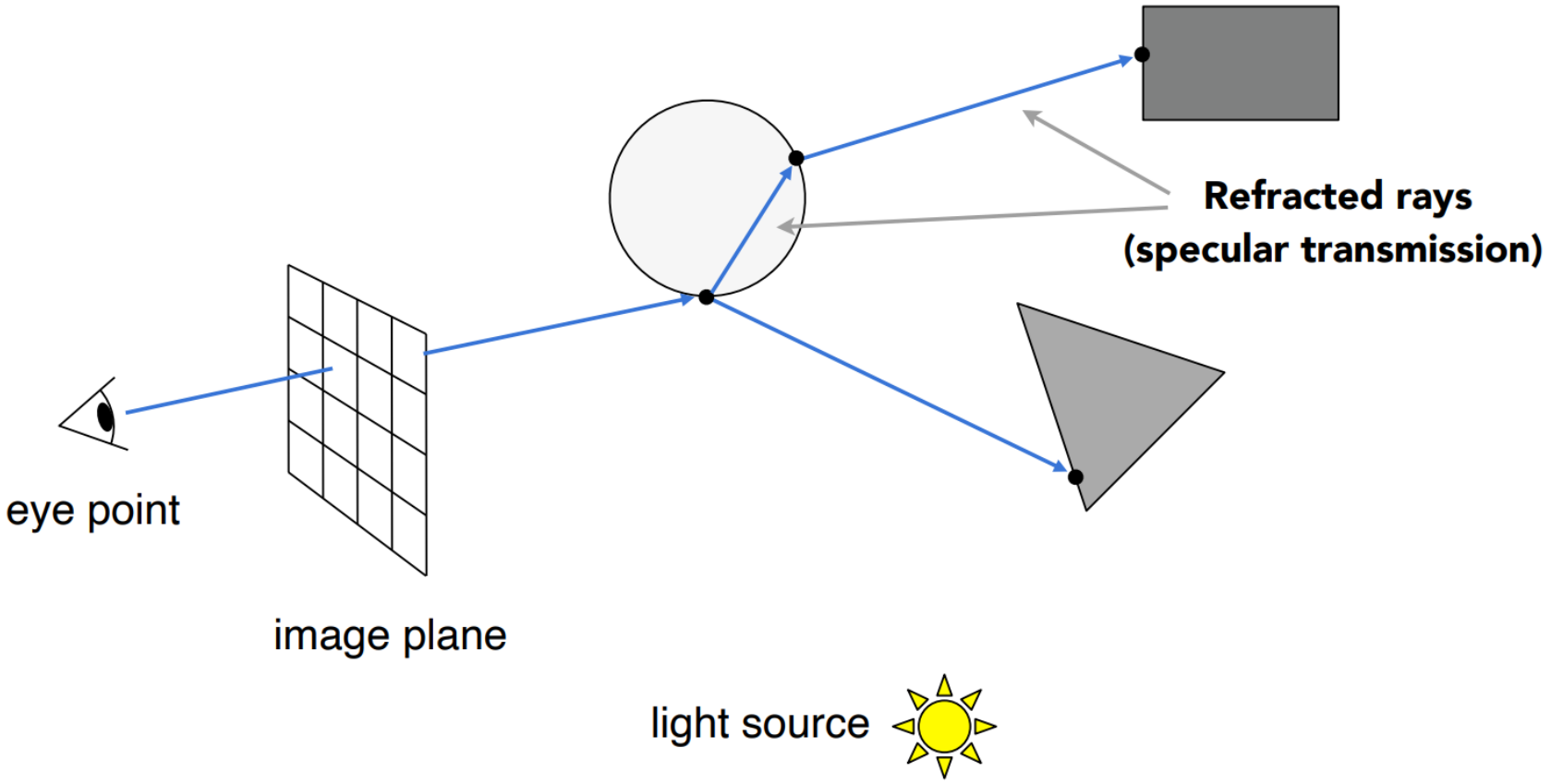
Recursive Ray Tracing



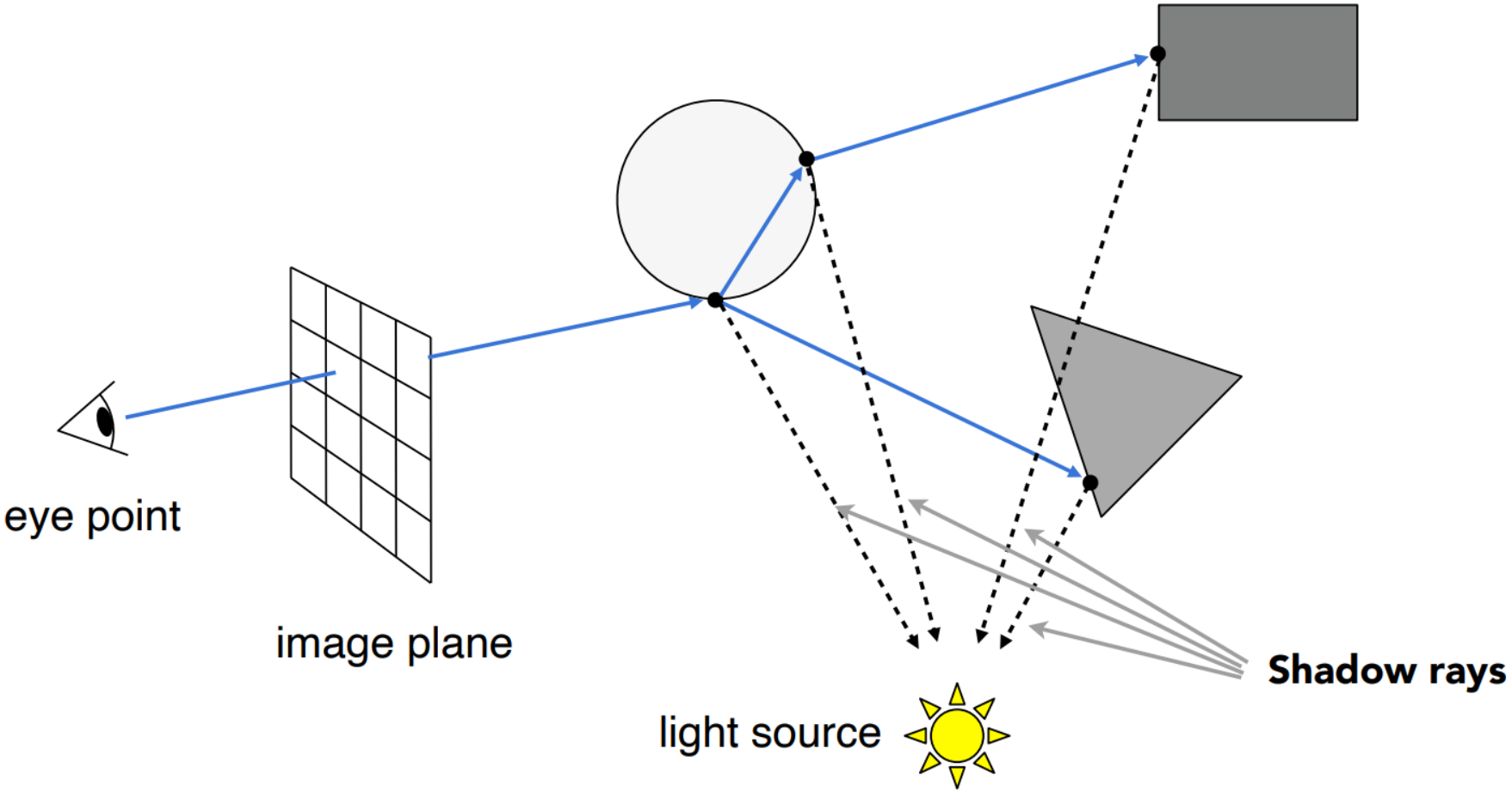
Recursive Ray Tracing



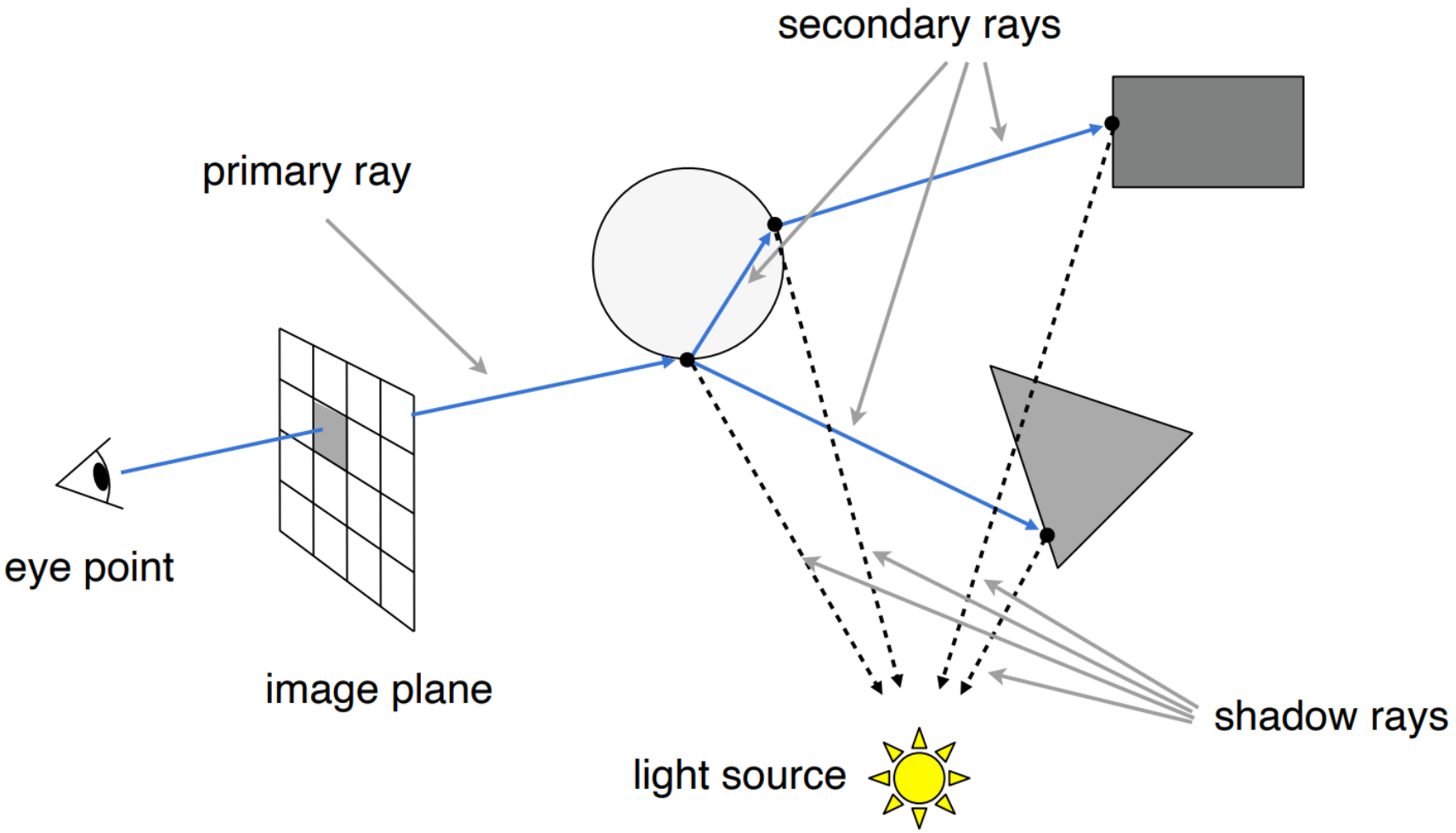
Recursive Ray Tracing



Recursive Ray Tracing



Recursive Ray Tracing



递归的终止条件

- 在每个相交时，有些光线被吸收
  - 跟踪余下的量
- 忽略进入无穷空间的光线（或背景光强）
  - 在场景周围放一个大球
- 递归步数有限制

算法框架

```
color trace(point p, vector d, int step)
{
   color local, reflected, transmitted;
   point q;
   normal n;
   if(step > max) return(background_color);

   q = intersect(p,d,status);
   if(status == light_source)
          return(light_source_color);
   if(status == no_intersection)
          return(background_color);

   n = normal(q);
   r = reflect(q,n);
   t = transmit(q,n);
   local = phong(q,n,r);
   reflected = trace(q,r,step+1);
   transmitted = trace(q,t,step+1);

   return(local+reflected+transmitted);
}
```

Recursive Ray Tracing

计算效率

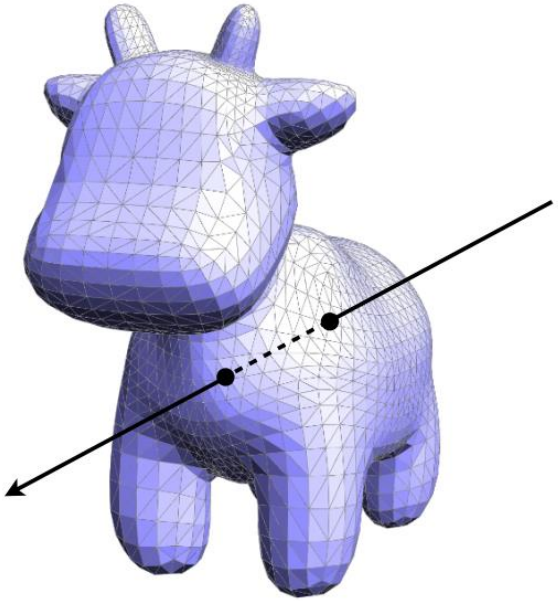
计算效率不高！

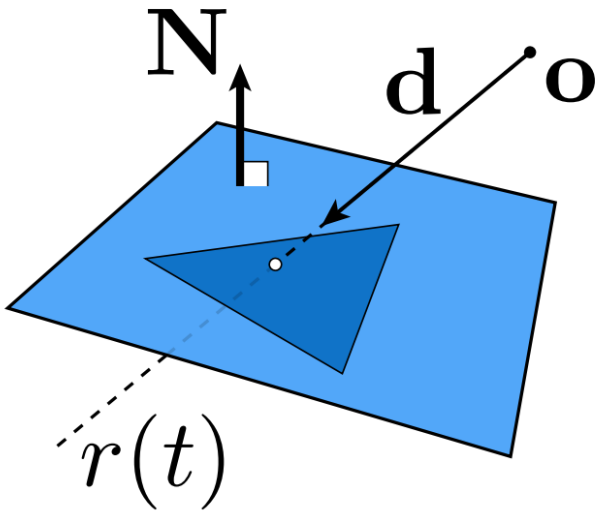
慢

大量的求交计算

- 点与三角网格的求交
- 点与三角形的求交

向量运算!

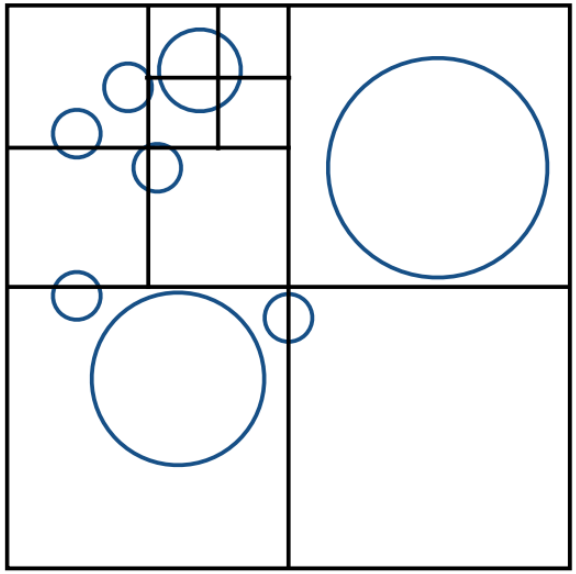




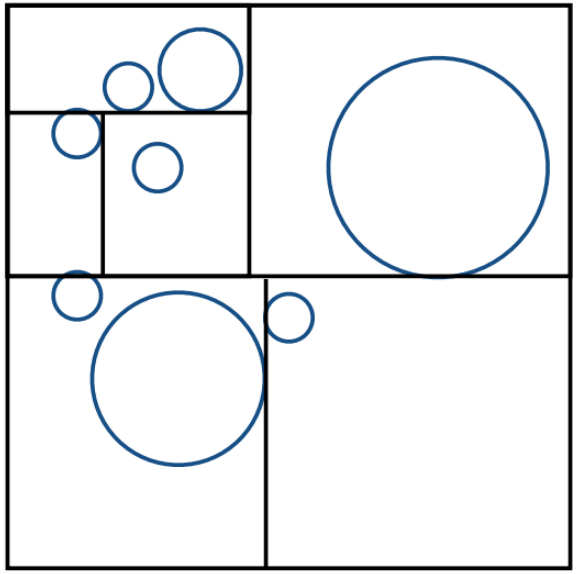
加速策略： Bounding Volumes

- Axis-Aligned Bounding Box (AABB) (轴对齐包围盒)
- 包围球

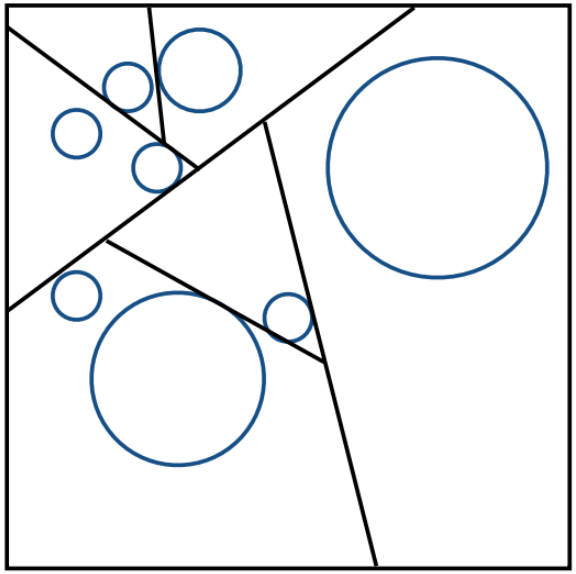
加速策略：空间组织



Oct-Tree

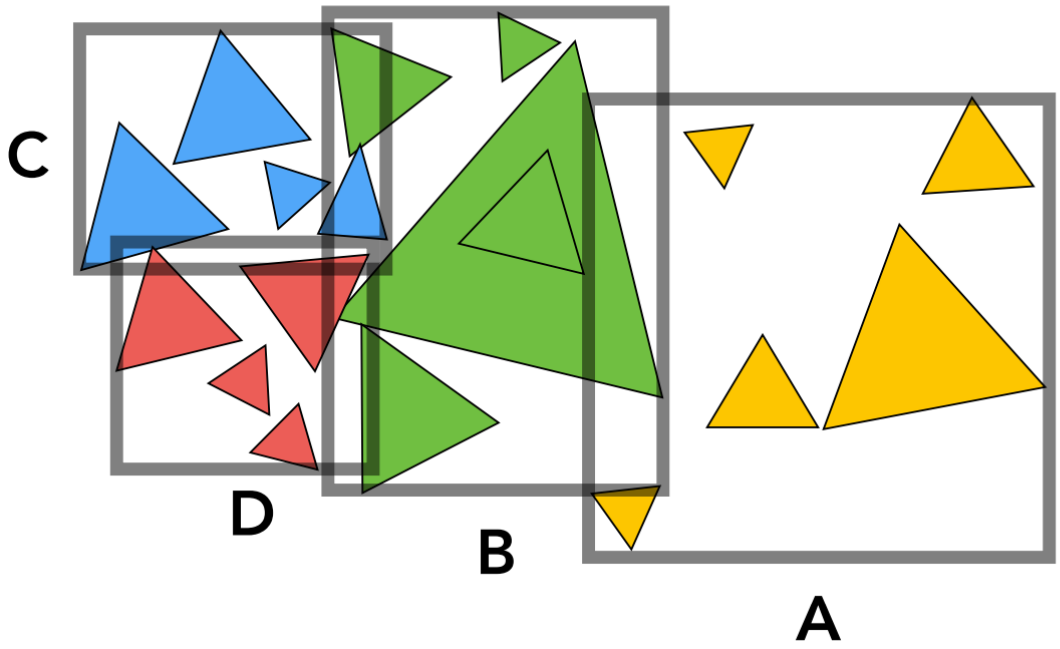


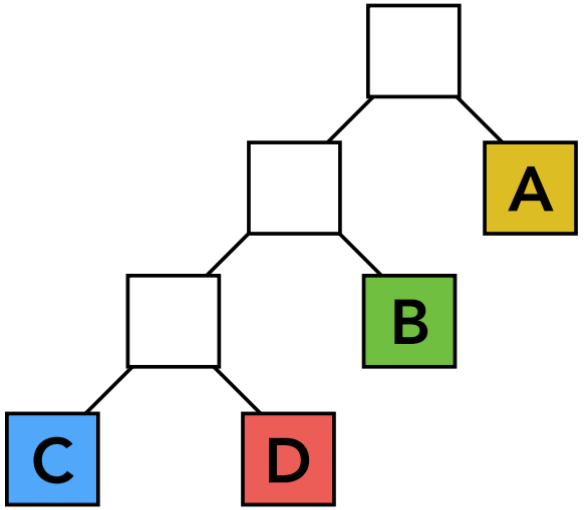
KD-Tree



BSP-Tree

加速策略：Bounding Volume Hierarchy (BVH)

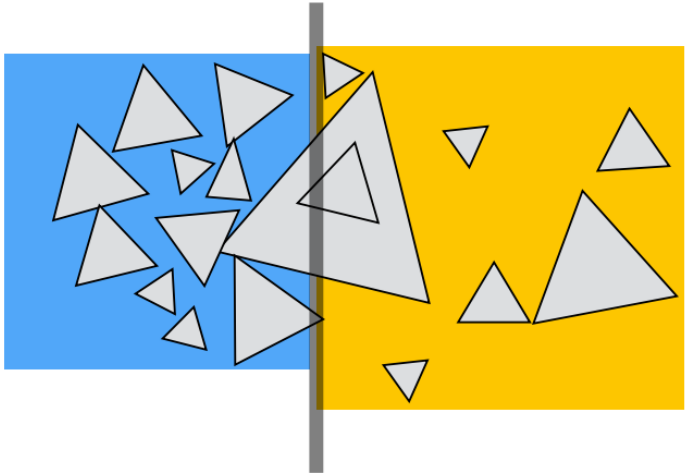




Spatial vs Object Partitions

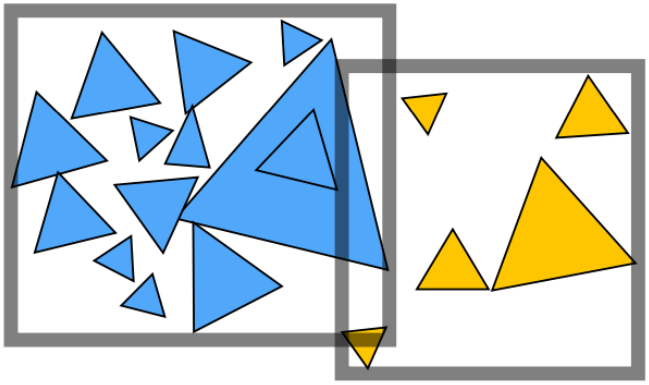
Spatial partition (e.g.KD-tree)

- Partition space into non-overlapping regions
- An object can be contained in multiple regions



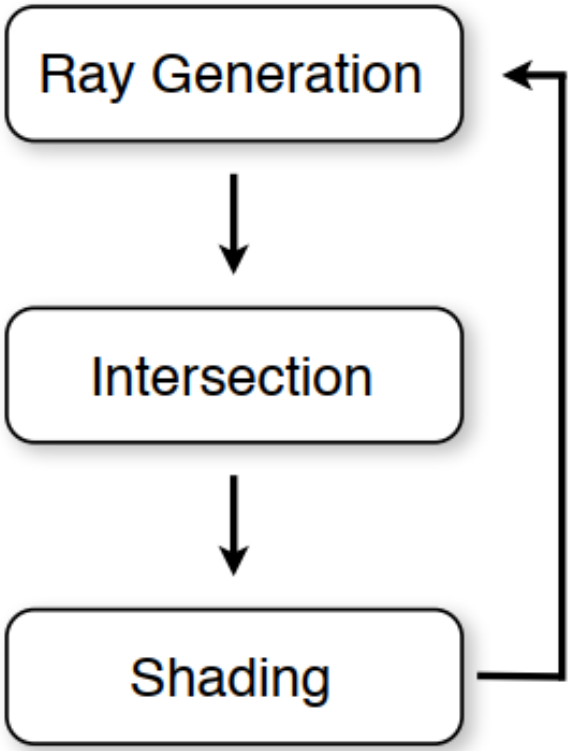
Object partition (e.g. BVH)

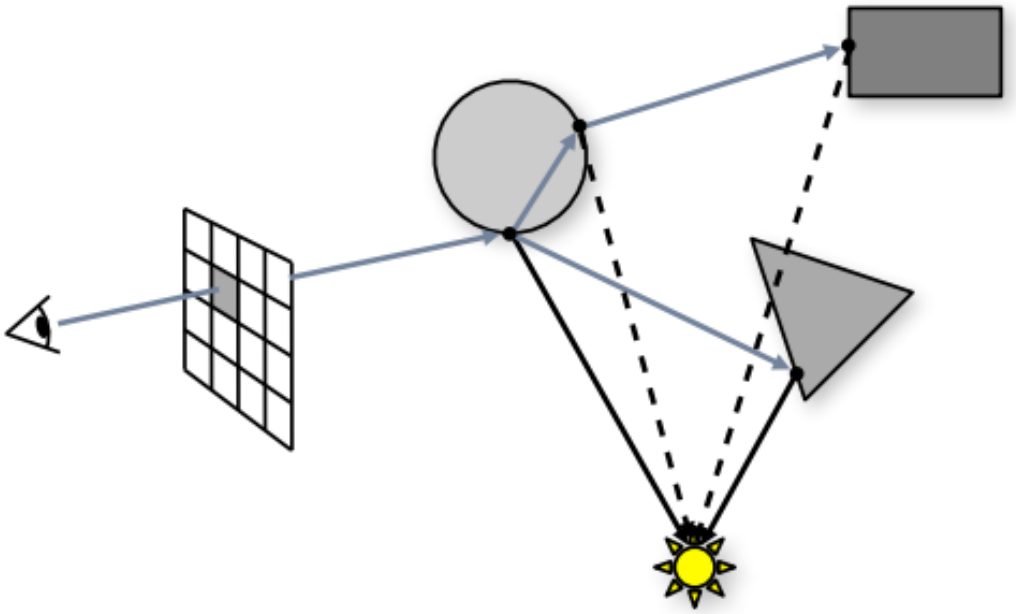
- Partition set of objects into disjoint subsets
- Bounding boxes for each set may overlap in space



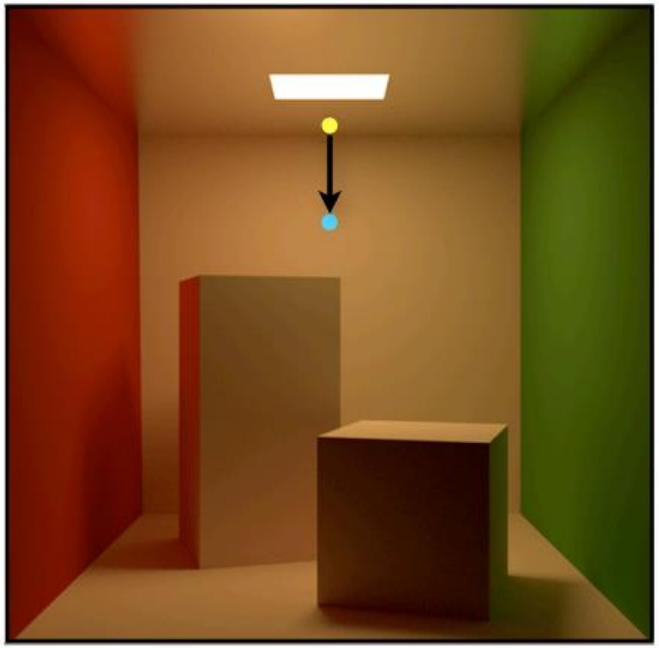
Discussions

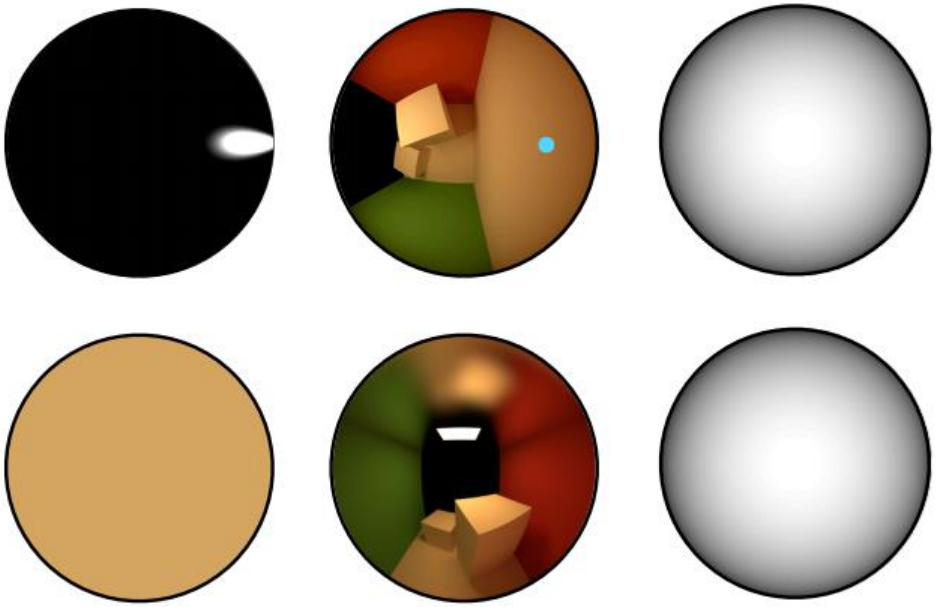
Recap: Ray Tracing



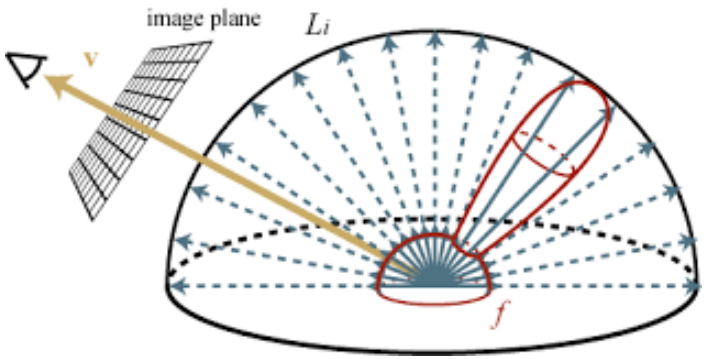


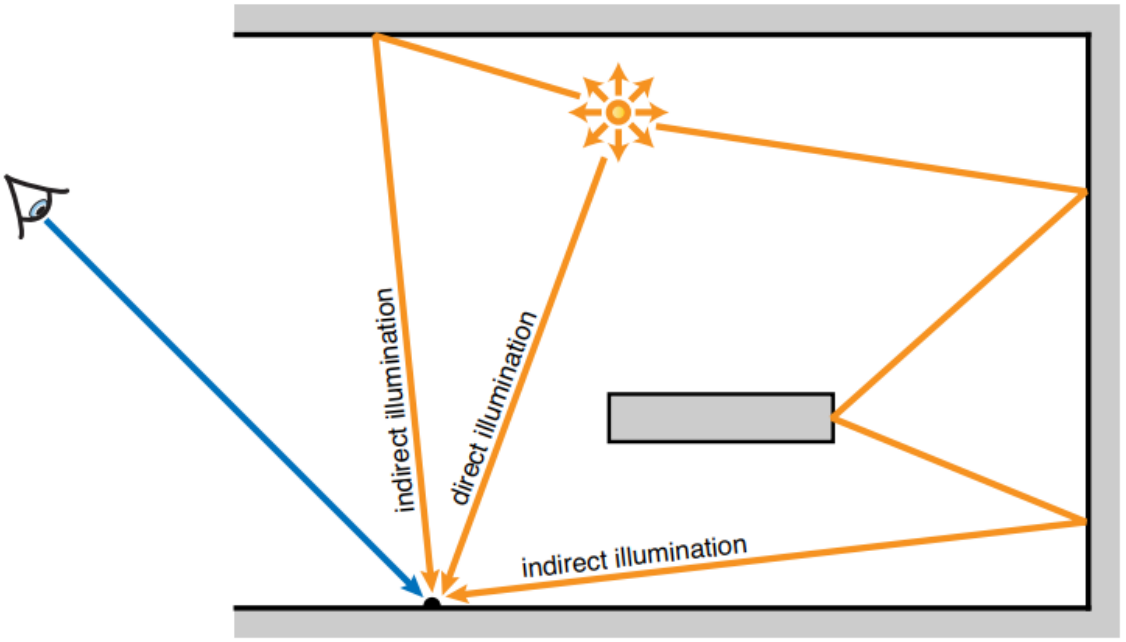
$$L(\mathbf{x},\vec{\omega}) = L_e(\mathbf{x},\vec{\omega}) + \int_{H^2} f_r(\mathbf{x},\vec{\omega}',\vec{\omega}) L(r(\mathbf{x},\vec{\omega}'),-\vec{\omega}') \cos\theta' d\vec{\omega}'$$





光线跟踪算法是在求解渲染方程吗？

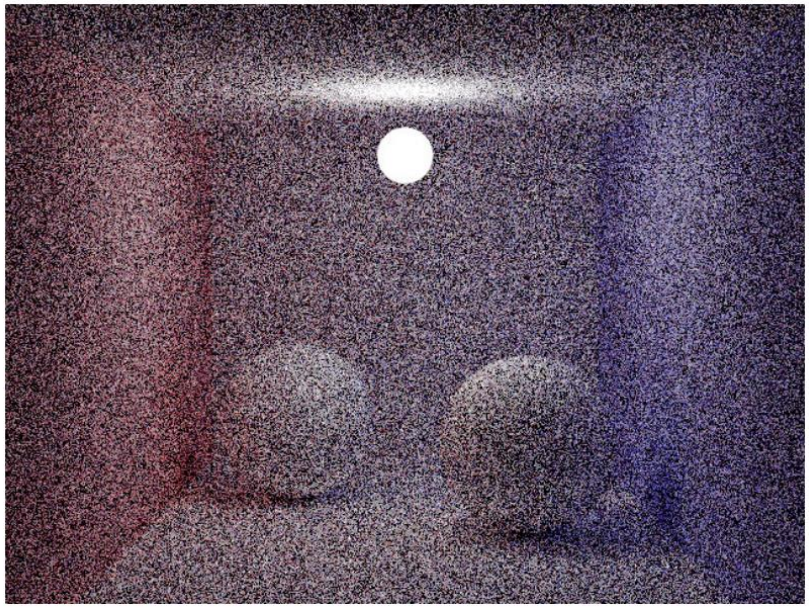




Whitted光线跟踪算法只是计算了一条反射光路，而不是整个半球面

Challenge: Noise?

- Small light source leads to high noise
  - Because the probability for a “light path” to hit the light source is small

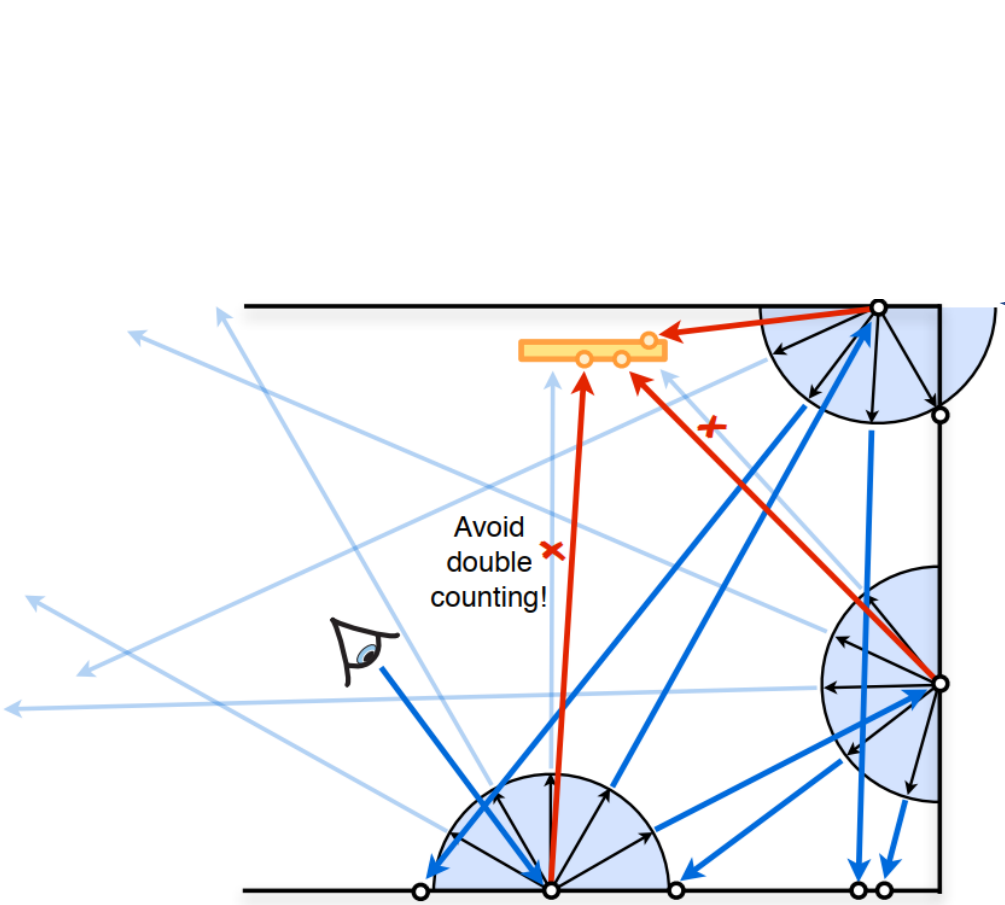


64 samples per pixel

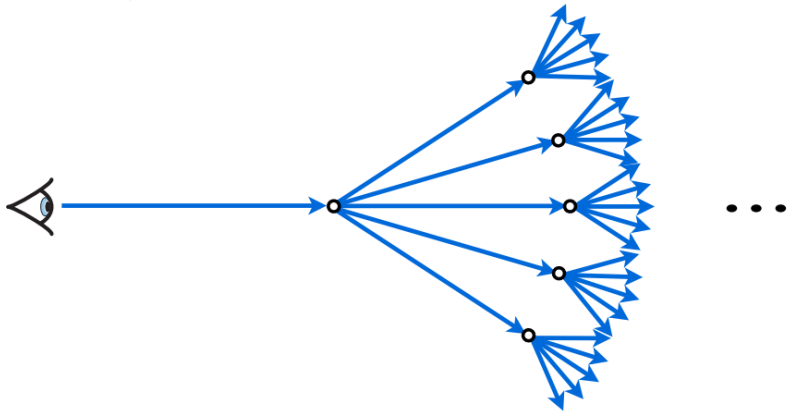
Sampling Problem!

如何真正求解渲染方程？

- 积分方程的求解，如何计算积分？

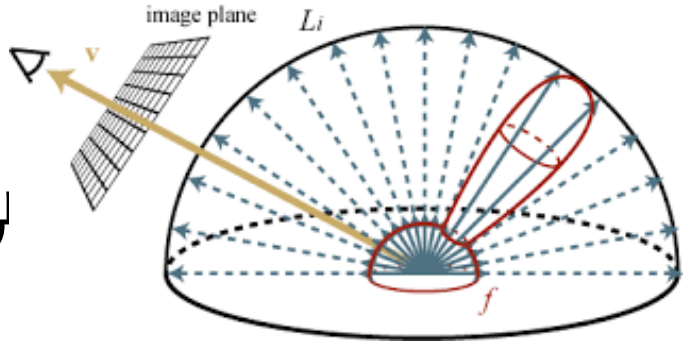


光线数爆炸！



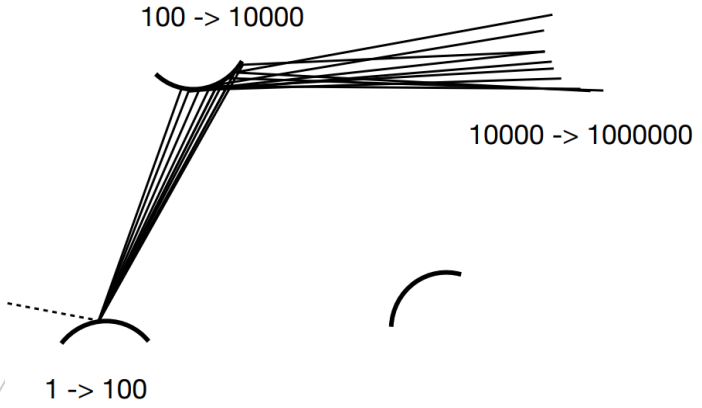
如何求积分？

$$L_o(p,\omega_o) = L_e(p,\omega_o) + \int_{\Omega+} L_i(p,\omega_i) f_r(p,\omega_i,\omega_o)(n\cdot\omega_i)\,\mathrm{d}\omega_i$$



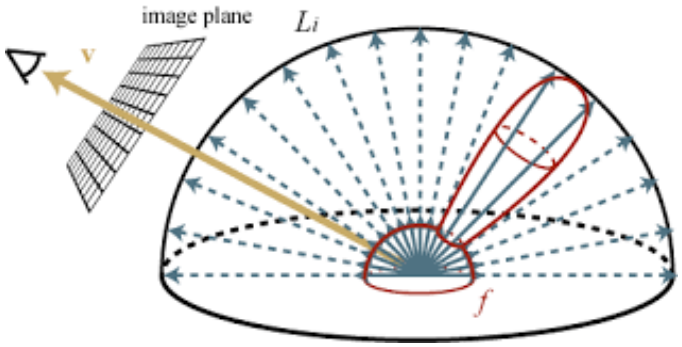
- 在半球面均匀/随机采样N条光
  - 光线数指数增长！！

随机采样1条光线，
多次计算！



Path Tracing: Basic Idea

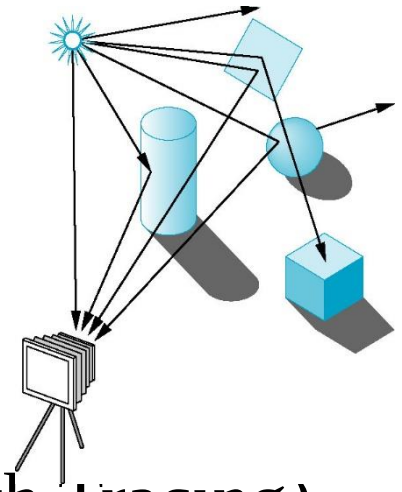
$$L_o(p,\omega_o) = L_e(p,\omega_o) + \int_{\Omega+} L_i(p,\omega_i) f_r(p,\omega_i,\omega_o)(n\cdot\omega_i)\,\mathrm{d}\omega_i$$



- 从视点往像素打N条光线
- 每条光线生成一条光路(path)，计算着色
  - 每点按概率只产生一个反射方向
- 将所有光路的着色进行平均
  - 大数定理（当N趋向无穷）

Ray Tracing Methods

- Ray Casting (光线投射) [Appel 1968]
  - 是Ray tracing的第一步，发射光线与物体相交
- Ray tracing (光线跟踪/追踪)
  - 一个框架，不是个具体的方法
  - Recursive ray tracing [Whitted 1980]
  - Stochastic ray tracing [Cook et al. 1984]
- Path tracing (路径追踪)
  - Ray tracing + 蒙特卡洛法 [Veach 1997]
    - http://graphics.stanford.edu/papers/veach_thesis/
- Advanced Light Transport (Advanced Path Tracing)



谢谢！

Thank You